



What characteristics make ChatGPT effective for software issue resolution? An empirical study of task, project, and conversational signals in GitHub issues

Ramtin Ehsani¹ · Sakshi Pathak¹ · Esteban Parra² · Sonia Haiduc³ · Preetha Chatterjee¹

Received: 28 January 2025 / Accepted: 29 September 2025 / Published online: 18 November 2025
© The Author(s) 2025

Abstract

Conversational large-language models (LLMs), such as ChatGPT, are extensively used for issue resolution tasks, particularly for generating ideas to implement new features or resolve bugs. However, not all developer-LLM conversations are useful for effective issue resolution and it is still unknown what makes some of these conversations not helpful. In this paper, we analyze 686 developer-ChatGPT conversations shared within GitHub issue threads to identify characteristics that make these conversations effective for issue resolution. First, we empirically analyze the conversations and their corresponding issue threads to distinguish helpful from unhelpful conversations. We begin by categorizing the types of tasks developers seek help with (e.g., *code generation*, *bug identification and fixing*, *test generation*), to better understand the scenarios in which ChatGPT is most effective. Next, we examine a wide range of conversational, project, and issue-related metrics to uncover statistically significant factors associated with helpful conversations. Finally, we identify common deficiencies in unhelpful ChatGPT responses to highlight areas that could inform the design of more effective developer-facing tools. We found that only 62% of the ChatGPT conversations were helpful for successful issue resolution. Among different tasks related to issue resolution, ChatGPT was most helpful in assisting with code generation, and tool/library/API recommendations, but struggled with generating code explanations. Our conversational metrics reveal that helpful conversations are shorter, more readable, and exhibit higher semantic and linguistic alignment. Our project metrics reveal that larger, more popular projects and experienced developers benefit more from ChatGPT's assistance. Our issue metrics indicate that ChatGPT is more effective on simpler issues characterized by limited developer activity and faster resolution times. These typically involve well-scoped technical problems such as compilation errors and tool feature requests. In contrast, it performs less effectively on complex issues that demand deep project-specific understanding, such as system-level code debugging and refactoring. The most common deficiencies in unhelpful ChatGPT responses include incorrect information and lack of comprehensiveness. Our findings have wide implications

Communicated by: David Lo.

Extended author information available on the last page of the article

including guiding developers on effective interaction strategies for issue resolution, informing the development of tools or frameworks to support optimal prompt design, and providing insights on fine-tuning LLMs for issue resolution tasks.

Keywords Issue resolution · Large language models · GitHub · Conversation analysis

1 Introduction

Conversational Large Language Models (LLMs) have emerged as crucial tools for providing software development support. According to GitHub's 2023 Octoverse report, more than 92% of developers use AI-based coding tools, including ChatGPT (GitHub 2024). The emergence of these models has changed the way developers seek help with issue resolution, potentially replacing traditional platforms such as Stack Overflow (Eastman 2023). Conversational LLMs are particularly valued in problem-solving tasks like issue resolution because of their ability to understand and generate human-like dialogue, thereby supporting an interactive learning experience (Ross et al. 2023).

Developers widely use conversational LLMs for brainstorming new features or resolving bugs (Hou et al. 2024; Wu et al. 2023). ChatGPT conversation links are shared in GitHub issue threads and pull requests to discuss potential solutions, support claims, and illustrate examples (Fan et al. 2023; Tang et al. 2023; Hao et al. 2024; Chouchen et al. 2024). However, these LLM responses can be of low quality or contain incorrect information (Li et al. 2023; Das et al. 2024; Zhou et al. 2024; Liu et al. 2024). These inaccuracies are hard for developers to detect (Kabir et al. 2024), potentially leading to poor design decisions and software flaws, ultimately resulting in ineffective issue resolution.

There is a growing body of research on using LLMs for issue resolution with a focus on areas like bug localization (Hossain et al. 2024), and program repair (Jimenez et al. 2024; Zhang et al. 2024). Prior research has highlighted the importance of developer conversations in successful issue resolution (Chatterjee et al. 2020; Kavalier et al. 2017). However, detailed analysis of conversations between developers and LLMs for issue resolution remains limited. To address this gap, our previous work (Ehsani et al. 2025b) examined 433 developer-ChatGPT conversations in GitHub issue threads, examining how knowledge gaps in prompts and conversation styles affect issue closure. Over 54% of prompts exhibited knowledge gaps, including *Missing Context*, *Missing Specifications*, *Multiple Contexts*, and *Unclear Instructions*. While we identified critical knowledge gaps in developer prompts, our analysis was limited to understanding what gaps exist and their frequency. We did not explore the broader interaction dynamics, such as *how* developers adapt to these challenges, *how* task type, project complexity, or communication patterns contribute to helpful or unhelpful outcomes, or *what* strategies could mitigate their impact during multi-turn interactions. Understanding which task and project settings are more likely to lead to helpful outcomes can provide developers with actionable insights for effective issue resolution, helping them both avoid common knowledge gaps and craft better prompts. Related research has also explored why developers engage in multi-turn conversations with ChatGPT and their motivations for sharing these conversations on GitHub (Hao et al. 2024). A recent work by Das et al. revealed that ChatGPT is primarily used for ideation rather than validation, with generated code rarely being used directly to resolve issues (Das et al.

2025). Mondal et al. identified gaps and factors that prolong these conversations (Mondal et al. 2024). While such conversations do not always lead to successful outcomes, prior studies have not investigated the specific characteristics that make some developer-LLM conversations helpful and others not, or how they contribute to effective issue resolution.

In this paper, we aim to address these gaps by analyzing a dataset of 686 issue-related conversations between developers and ChatGPT shared on GitHub issues. Our goal is to gain insights into the types of tasks related to issue resolution that ChatGPT is effective in assisting, and understanding the communication patterns and project characteristics that are associated with successful issue resolution using ChatGPT. Additionally, we identify the common deficiencies in unhelpful ChatGPT responses. By exploring the dynamics of developer-ChatGPT interactions, including how prompt characteristics (code and code-related information) and interaction strategies influence successful outcomes, we aim to uncover actionable insights to improve LLM-based issue-resolution workflows. Specifically, we investigate the following research questions:

RQ1 *How helpful are ChatGPT conversations in successfully resolving issues?* Using a combination of textual cues and GitHub metadata, we identify developer-ChatGPT conversations that are helpful in resolving issues and those that are not. Only 62% of the conversations were identified as helpful. We also identify the specific tasks for which developers sought assistance during these conversations. ChatGPT was more helpful with tasks of code generation and tool/library/API recommendations, while it struggled with generating code explanations and providing up-to-date information.

RQ2 *What are the key characteristics of issue-resolution conversations with ChatGPT? Do these characteristics vary between helpful and unhelpful conversations?* Using a set of conversational, project, and issue-related metrics, we investigate what differentiates helpful conversations from unhelpful ones. The conversational metrics are inspired by previous research on how developers ask questions on platforms like StackOverflow (Chatterjee et al. 2021; Duijn et al. 2015). Our goal is to understand the challenges developers face in LLM-guided issue resolution and design metrics that could be potentially leveraged to build developer support tools. We found that in helpful conversations, developers provide clear, well-structured instructions and minimize topic changes within the same conversation thread. Our project-related metrics provide insights into how project size, complexity, and popularity affect ChatGPT's helpfulness. We observed that larger projects and experienced developers are able to use ChatGPT more effectively for issue resolution. Our issue-level metrics are designed to capture the underlying complexity and nature of the issues themselves. We found that ChatGPT is more helpful on focused, lower-complexity issues that are resolved quickly and involve fewer developers. In contrast, it tends to struggle with complex, time-consuming issues that require high collaboration and deep project-specific knowledge.

RQ3 *What are the common deficiencies in ChatGPT's responses in unhelpful issue resolution conversations? How do they vary across different tasks?* We found that in unhelpful issue resolution conversations, ChatGPT often produces incorrect or non-functional code, sometimes suggesting bad practices or even system-breaking solutions. Additionally, it generates unclear answers that confuse developers and hallucinates incorrect information, referencing non-existent facts or functions in various libraries.

We summarize below our key contributions in this paper:

- We introduce the first manually annotated dataset of helpful and unhelpful developer-ChatGPT issue resolution conversations, consisting of 876 conversations collected from 655 projects, with 686 of the conversations from 543 projects being directly related to issues. Our code and data are made publicly https://github.com/SOAR-Lab/llm_issue_resolution_analysis available¹.
- We present the first in-depth analysis of helpful vs. unhelpful developer-ChatGPT conversations for issue resolution. We identify conversational, project, and issue metrics associated with helpful conversations. We also analyze how ChatGPT performs across different issue resolution-related tasks.
- We identify common deficiencies that ChatGPT exhibits in unhelpful responses related to issue resolution.
- We offer actionable strategies for developers to help developers obtain useful ChatGPT suggestions for issue resolution. Our findings can also inform design of prompt optimization tools and fine-tuning LLMs for issue resolution.

2 Dataset Collection and Preprocessing

In May 2023, OpenAI introduced a feature allowing users to share their conversations with ChatGPT through dedicated links (OpenAI 2023). Using this feature, Xiao et al. collected a set of developer-ChatGPT conversations publicly shared on GitHub until October 2023, forming the DevGPT dataset (Xiao et al. 2024). The size of this dataset is limited, containing only 413 conversations collected from 2023-01-01 to 2023-10-13. We expand this dataset by collecting additional data using GitHub’s API endpoint, searching for occurrences of ChatGPT links in all GitHub issues. The data is collected from 2023-01-01 to 2024-04-30, encompassing the same window as DevGPT and extending it by more than six months. We gathered all publicly available developer-ChatGPT conversation links, resulting in 876 unique conversation links from 823 GitHub issues across 655 repositories, as shown in Table 1. Each issue thread can contain more than one link to developer-ChatGPT conversations. This dataset is not a sampled subset, but rather the result of an exhaustive crawl of all publicly shared conversation links found in GitHub issues. While prior studies relied on the DevGPT dataset (Ehsani et al. 2025b; Mondal et al. 2024), our augmented dataset more than doubles the size and offers significantly broader temporal and project-level coverage. Table 2 provides a side-by-side comparison of the original DevGPT dataset and our extended dataset, highlighting improvements in temporal coverage, issue count, and repository diversity.

We filtered out non-English conversations using Python’s *lingua-py* library (Stahl 2024). We replaced code snippets with [CODE] and stack traces/error messages with [ERROR] inside the conversations to focus our analysis on the text rather than code and errors; the text embeds the information we are looking for, such as the description of the task, discourse style, etc. ChatGPT-generated responses structure code snippets within quote blocks, simplifying their replacement using RegEx. However, prompts are often not structured in this manner. LLMs have shown promise in automatically detecting code from unstructured

¹ https://github.com/SOAR-Lab/llm_issue_resolution_analysis

Table 1 Details of Our Extended Dataset

Unique Projects	Unique Issues	ChatGPT Links	State of Issues	
655	823	876	#Open	408
			#Closed	415

Table 2 Comparison Between DevGPT and Our Extended Dataset

Dataset	Time Span	Issues	Repositories	Conversations
DevGPT	Jan-Oct 2023	388	300	413
Our Dataset	Jan 2023-Apr 2024	823	655	876

documents (Oedingen et al. 2024). Therefore, we used *GPT-4* to identify code snippets and error messages in developer prompts. To ensure the effectiveness of this method, one of the authors then conducted a manual validation for the entire dataset.

3 Methodology

3.1 RQ1. How Helpful are the Conversations with ChatGPT in Successfully Resolving Issues?

We qualitatively analyzed the issue threads containing developer-ChatGPT conversations to answer RQ1, focusing on identifying the **indicators** that can help us determine the helpfulness of a ChatGPT conversation in resolving an issue and the **types of tasks** related to issue resolution presented to ChatGPT. To determine whether a developer-ChatGPT conversation was *helpful* or *unhelpful*, we examined the issue thread discussions and GitHub metadata. Specifically, we identified three types of helpfulness indicators: a) textual cues in issue threads, b) appreciation emojis in issue threads, and c) direct implementation via subsequent pull requests and/or commits. These indicators emerged from inductive coding after qualitatively reviewing 100 randomly chosen issue threads. Two annotators analyzed all available metadata (e.g., comments, commits, pull requests) to identify consistent and observable signals of helpfulness. These three criteria were the only robust and repeatable markers of helpfulness across the dataset.

Textual Cues We identified two types of textual cues.

- *Explicit Positive Comments on ChatGPT Suggestions:* Developers explicitly praise or acknowledge the helpfulness, relevance, or accuracy of ChatGPT’s contributions inside the issue thread. Notable indicators of positive feedback included remarks such as “*Chat-GPT made fast work of this one :P*” or “*I should add GPT to my debug toolbox.*”
- *Using ChatGPT Suggestions For Further Exploration:* This occurred when ChatGPT’s suggestions were indirectly used in two ways: **(a)** Issue Discovery and Planning - information from ChatGPT helped identify issues, plan future actions and features, or explore potential solutions. For instance, comments as the following reflect proactive planning based on ChatGPT insights: “*...here’s an example of a problem that GPT-4*

can easily one-shot: [Link]. In an ideal world, [name of package] could handle making the necessary edits... [Another developer] I put together a proof of concept for this: [URL]". (b) Idea Generation and Brainstorming - ChatGPT guided problem-solving and brainstorming sessions. For instance, comments such as "...I've been studying how to get this done, these links may be useful...[Link]" indicate ChatGPT's use for idea generation and exploration.

Appreciation Emojis We identified developers' reactions to the shared ChatGPT links via emojis (e.g., hearts or likes), indicating user appreciation.

Direct Implementation We assessed helpfulness by investigating if ChatGPT's suggestions were directly applied to resolve issues. We examined whether developers made commits or shared links to pull requests integrating the suggested code or idea, after the link was shared within the issue thread. This process involved a manual, line-by-line analysis of the pull request or commit content, cross-referenced with the specific responses provided by ChatGPT. We looked for clear alignment, including identical or near-identical code snippets, reused logic, distinctive variable or function names, and structural similarities in proposed fixes or strategies. To ensure methodological rigor, the same two authors collaboratively conducted the analysis across all candidate cases. For each instance of potential direct implementation, both authors reviewed the surrounding context (including timestamps, developer comments, and code diffs) to determine whether the changes could be attributed to the ChatGPT suggestions. Any disagreements were resolved through multiple rounds of discussion, during which both authors revisited the evidence, clarified interpretation criteria, and refined the decision logic. Final labels were assigned only after full consensus was reached.

The three types of explicit indicators mentioned above (e.g., textual cues, appreciation emojis, direct implementation) are not mutually exclusive. A single conversation may exhibit one or multiple forms of evidence (e.g., a developer may leave a positive comment and also implement the suggestion). We found 103 instances where code suggestions from ChatGPT lacked any of the three types of explicit indicators. In these cases, we check if the suggested code was directly integrated into the project's codebase using a code clone detection tool. We extracted subsequent commits made to each project's Git repository after the date when the link was shared in the thread and used the code clone detection tool NiCad Version 6.2 (Cordy and Roy 2011) on each commit, comparing it to the code provided by ChatGPT. To extend NiCad's support for additional languages, we added grammars based on the TXL grammar resources following their documentation². If the tool identified a similarity between the code provided by ChatGPT and the commit made to the project, we mark the conversation as helpful. If no similarity was found, we assume that the conversation was unhelpful for the issue. Among these 103 instances, 87 instances were written in languages supported by NiCad, either natively or with added grammars and 16 instances of unsupported languages (e.g., YAML, JSON, TypeScript) for which no grammar files were available. The distribution of programming languages was as follows: Python (49), JavaScript (15), Java (9), C++ (5), Rust (4), C (2), C# (2), Swift (1), unsupported (16). The 16 cases in which the language was unsupported were labeled as unhelpful because no code similarity analysis could be performed and no explicit helpfulness indicators were

²<https://txl.ca/txl-resources.html>

present in the issue thread. For the rest, we used NiCad. The default threshold for code similarity is set to 30%, which is NiCad's default setting and has been widely adopted in prior studies for detecting meaningful code clone (Das et al. 2024). This threshold allows for minor edits, such as renaming variables or reformatting, while still capturing structural similarity. Given our goal of identifying cases where developers incorporated ChatGPT's suggestions, this level of sensitivity was appropriate for our analysis.

All these indicators are designed to capture different forms of evidence and avoid relying on any single signal. Developers may find a response helpful without expressing explicit gratitude. To account for these variations, we combined different complementary indicators. This approach allows us to cover a wider range of helpfulness signals and reduces the likelihood of misclassification.

Unhelpful ChatGPT conversations are those where either the links receive negative reactions (textual or emojis) from developers, or the corresponding issue threads do not contain subsequent pull requests or commit links, and no code similarity between the suggested code and the project codebase is detected using NiCad.

To determine *the types of SE tasks* presented to ChatGPT related to issue resolution, we analyzed the data in multiple phases. As part of this process, the annotators did not rely solely on the content of the ChatGPT conversation. They examined the broader GitHub issue thread, specifically, all comments, updates, and discussions leading up to the point where ChatGPT was invoked, to understand the context in which ChatGPT was used and to infer the task it was asked to help with. This allowed us to situate each task within the overall flow of the issue thread and capture how ChatGPT fits into real-world issue resolution practices.

Two authors first reviewed 100 issue threads containing ChatGPT conversation links and used open coding (Corbin and Strauss 2008) to identify the SE tasks. Both annotators have 3+ years of experience in programming and qualitative analysis. Up to two tasks were identified for each data point, as developers often had multiple requests in the same conversation. Analysis of the first 100 threads resulted in the identification of 12 *types of tasks*. These categories started as high-level tasks such as code-related tasks or library debugging, and got merged and refined throughout the annotation process. For example, *library recommendation* and *library usage queries* were merged. In each of the next two phases, an additional 100 data instances (200 more data points in total) were examined independently by the two authors and discussed in iterative discussions (two in each phase) to resolve conflicts and refine the coding scheme, resulting in a final set of 9 tasks, as shown in Table 3. Conflicts were resolved collaboratively through consensus between the annotators. Cohen's Kappa inter-rater agreement was calculated at each stage, reaching 0.89, a threshold supported by literature as strong for high-quality annotations (McHugh 2012). Once this level of agreement was achieved, the remaining data was divided between the two annotators for separate labeling, maintaining the consistency established earlier.

We found that 190 conversations were unrelated to issue resolution. For instance, developers shared links with comments like "*I have ChatGPT Plus. After trying to use this link, it gives me a 404*". We categorized such conversations as *miscellaneous* and discarded them from further analysis. We report our findings on the remaining 686 relevant conversations, shared within issue threads of 543 GitHub projects. This remaining dataset was labeled using the nine categories described in Table 3. Mind that the sum of the numbers

Table 3 Types of Tasks Presented to ChatGPT

Category	Definition	Example	Freq.
Code Generation and Implementation	Generation of code artifacts and scripts such as classes, functions, or modules within a project	<i>"Create a bash script to perform server shutdown procedures"</i> (GitHub 2023e)	196
Tool/Library/API Recommendation	Inquiries about the usage, functionality, or integration of specific libraries, APIs, or tools	<i>"I'm using TouchableOpacity in React, but opacity is lightened even when the user is dragging a list [...] Why is this happening?"</i> (GitHub 2023x)	182
Bug Identification and Fixing	Debugging programs, analyzing stack traces, repairing code, and modifying code to fix bugs or errors	<i>"Hi I'm getting these issues with fonts in CSS [stack trace]"</i> (GitHub 2023a)	120
SE-Information-Seeking	Seeking information on specific or general knowledge about SE concepts or practices	<i>"What is Hacker News, what can you tell me about the people that frequent it?"</i> (GitHub 2023o)	111
Code Enhancement	Modifying a given code snippet to add more functionalities	<i>"I'd like to change it [code] so that it scales values to an optionally user-specified"</i> (GitHub 2023c))	77
System Design and Architecture	Discussions about designing websites, applications, or systems, as well as recommendations for system components, package/server, and GitHub configurations	<i>"There are 3 environments: test, development, and production. I would like to add an 'integration' environment. What would be the recommended way?"</i> (GitHub 2023v)	33
Code Explanation	Asking for explanation of a given code inside the conversation	<i>"I have never used html canvas before, [code], explain it to me"</i> (GitHub 2023d)	26
Test Generation	Generating unit tests and test pipelines	<i>"Now come up with a bunch of examples...Then turn those into pytest tests"</i> (GitHub 2023w)	5
Code Comment Generation	Generating comments for a given code	<i>"Write docstrings for each method"</i> (GitHub 2023b)	4

on the last column in Table 3 adds up to more than 686, which is the number of issue-related links in our dataset after the removal of unrelated links, because, as mentioned before, a conversation can have more than one category assigned to it.

3.2 RQ2. What are the Key Characteristics of Issue-Resolution Conversations with ChatGPT? Do they Vary Across Helpful and Unhelpful Conversations?

We quantitatively analyze developer-ChatGPT conversations based on three broad categories of metrics: Conversational, Project, and Issue Metrics. Each of these metric categories has several subcategories, as discussed in detail below.

3.2.1 Conversational Metrics

We analyze the *Structural*, *Linguistic*, and *Discourse* characteristics of the conversations. These metrics and their overall structure are shown in Fig. 1.

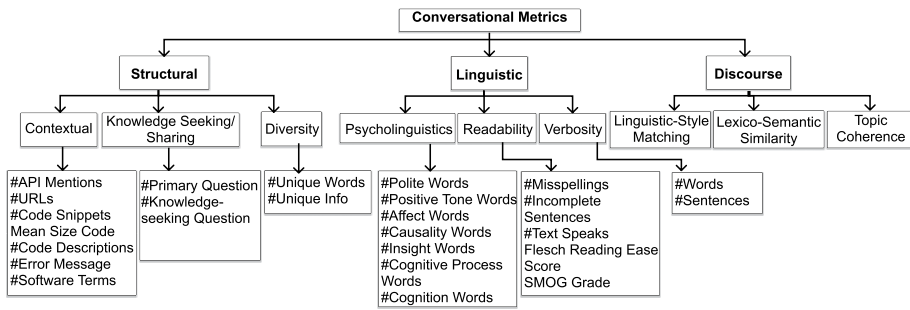


Fig. 1 Overall Structure of Conversational Metrics

Structural We use structural metrics to understand the types of questions developers ask, and the content they share with ChatGPT for seeking help with issue resolution. These metrics are adopted from Chatterjee et al.’s work on understanding the quality of developer chats (Chatterjee et al. 2021). We focus on two metric categories: Knowledge-seeking and Contextual.

Knowledge-Seeking metrics reveal the intent behind the questions developers ask ChatGPT. First, we identify the total number of *#Primary questions* in the prompts using question marks and 5W1H words (Cong et al. 2008). Next, we identify *knowledge-seeking questions* by counting primary questions containing “what,” “where,” or “how” (Harper et al. 2009).

Contextual metrics provide insight into the content and characteristics of the issues with which developers seek help. We measure several SE-related information in the conversations. First, we designed regular expressions for each programming language in our dataset (e.g., Python, Java, C#) to count *#API mentions* in the natural language. Next, we count *#URLs* or links using regular expressions. We also count *#Code snippets*, *#Error messages*, and calculate the *Mean size of code snippets* in a conversation. To measure *#Code descriptions*, we tokenize the code identifiers from the code snippets in each conversation and count the sentences mentioning these tokens (Chatterjee et al. 2021, 2017). We calculate the *#SE terms* in the text using a pre-compiled list of morphological terms and software-related terms (Chen et al. 2017; TechTerms 2024).

Linguistic We use linguistic metrics to assess the clarity, complexity, and overall communication effectiveness of the developer-ChatGPT conversations. We focus on four aspects: Psycholinguistics, Diversity, Readability, and Verbosity.

Psycholinguistic metrics study how the human mind processes and understands language, revealing the cognitive processes behind linguistic choices. Using LIWC, a psycholinguistic tool widely used in software engineering studies (Rigby and Hassan 2007; Han et al. 2024; Kabir et al. 2024; Zhou et al. 2023; Sandler et al. 2024), we measure *#Polite words*, *#Positive tone words*, and *#Affect words* to understand the emotional tone of the text. Additionally, we examine the *#Causality words*, *#Insight words*, *#Cognitive process words*, and *#Cognition words* to understand the cognitive and problem-solving processes behind developers’ issue resolution attempts, as exhibited in text.

Diversity metrics quantify the unique information within each conversation to understand how frequently developers introduce new information when interacting with ChatGPT. We use *#Unique words* and *#Unique info* as indicators of diversity. We first calculate

the number of unique words in the text, then determine unique information by calculating the ratio of distinct words to the total number of words (Chatterjee et al. 2021).

Readability metrics help us assess the ease of reading information in these conversations and whether it influences the effectiveness of ChatGPT in resolving issues. First, using Python's `pyspellchecker` (Barrus 2024), we count the *#Misspelled words* in a conversation. Using Python's package `spaCy` (spaCy 2024), we count the number of sentences without any subject or object as *#Incomplete* (Chatterjee et al. 2021). We measure the *#Text Speaks* in a conversation by using a list of 50 most common text speaks and abbreviations used in chats (Ponzanelli et al. 2014; Possel 2024). Next, we measure two popular reading score metrics: *Flesch Reading Ease Score* and *SMOG Grade*. We use Python's `py-readability-metrics` package to calculate these scores (Scott 2023).

Verbosity metrics measure the length of the conversations to analyze how the amount of communication impacts the conversation outcome. We calculate the *#Words* and *#Sentences*.

Discourse. We use discourse metrics to analyze how developers and ChatGPT adapt to each other's communication styles and maintain topic coherence throughout the conversation. We begin by analyzing standard conversation statistics, including the number of words, sentences, and conversation rounds. A conversation round refers to a single exchange consisting of a user prompt followed by a ChatGPT response. We also measure the sentiment patterns using *SentiStrength-SE* (Islam and Zibran 2018), a widely used tool made for sentiment analysis in software engineering contexts (Zhang et al. 2020). We use a software engineering-specific sentiment analysis tool because general-purpose models often misclassify technical expressions, whereas domain-specific tools such as *SentiStrength-SE* are tailored to capture sentiment nuances in developer conversations (Novielli et al. 2020; Imran et al. 2022). Overall, these metrics provide a basic understanding of conversational tone and engagement across helpful and unhelpful interactions. We capped our discourse metric-based analysis at conversation round 25, since only one unhelpful conversation in our dataset extended beyond this length. We focus on three key categories: Lexico-Semantic Similarity, Linguistic-Style Matching, and Topic Coherence.

Lexico-Semantic Similarity measures the semantic similarity between developer prompts and ChatGPT's answers throughout the conversation. We track how this score changes at each step to assess if the conversation stays semantically aligned. Using word embeddings from a pre-trained model optimized for semantic search (Reimers and Gurevych 2024), we compute the cosine similarity between the prompts and responses (Wang et al. 2021).

Linguistic-Style Matching (LSM) evaluates the similarity of linguistic style between developers and ChatGPT. We examine if developers and ChatGPT adjust their word choice and syntax to match each other at each step. We analyze each developer prompt with its corresponding ChatGPT answer and each prompt with the previous ChatGPT answer to track LSM changes. The score is calculated using 8 non-topical categories: articles, conjunctions, impersonal pronouns, personal pronouns, negations, prepositions, adverbs, and auxiliary verbs (Danescu-Niculescu-Mizil et al. 2011). A higher score indicates more LSM (Liu et al. 2022; Boyd 2022).

Topic Coherence refers to how the discourse evolves over several turns, maintaining focus on the same topic or diverging into others. (Vakulenko et al. 2018). To evaluate coherence in developer-ChatGPT conversations, we used topic modeling with Python's `Gensim` library (Röder et al. 2015). At each step, we analyzed the last two exchanges using Latent Dirichlet Allocation (LDA) to calculate semantic similarity between high-scoring words in the topics (Blei et al. 2003). For LDA, we used 1 topic and set the number of passes to 10,

as this configuration showed to yield stable topic distributions for our task. The decision to focus on the last two exchanges (steps) was deliberate. We wanted to capture any sudden shifts in the topic that might occur between consecutive prompt-answer pairs, as these shifts are key indicators of whether the conversation is drifting off-topic. Analyzing the entire conversation would obscure such shifts by aggregating too much data, potentially masking important transitions. A higher coherence score represents participants' focus on one technical topic and less drift to off-topic discussions.

3.2.2 Project Metrics

We analyze the *Repository Parameters* and *Developer Experience* characteristics of the projects involved in the conversations.

Repository Parameters We examine if the size, maturity, and popularity of the GitHub projects influence the perception of ChatGPT's helpfulness in issue resolution. To account for project size and maturity, we measure: *#Stars*, *#Contributors*, *#Forks*, *#Files*, and *#Lines of Code*. As indicators of project popularity we measure: *#Stars*, *#Contributors*, and *#Forks*. We extracted these metrics using GitHub's API, making specific calls for each project in our dataset. To count *#Files* and *#Lines of Code*, we cloned the repositories and ran the tool *cloc* (Danial 2021).

Developer Experience We identify the developers who first shared the links to the ChatGPT conversations in the issue threads and then examine if their level of experience influences the success of the conversations. We assume that the developer who first shares the link is the one who had the conversation with ChatGPT. Using GitHub's API, we extract the number of public repositories created by the developer (*#Public Repositories*), the number of followers the developer has (*#Followers*), the time difference between when the developer shared the link and when they first created their account (*Account Age*), and the total number of push commits and issue thread/pull request activities the developer has had on GitHub in the past year (*#Contributions*) from their public accounts.

3.2.3 Issue Metrics

For each conversation, we analyze the corresponding GitHub issue threads to investigate two key characteristics: *Issue Difficulty* and *Issue Type*. These characteristics help us examine whether certain types of issues or levels of complexity are associated with helpful or unhelpful ChatGPT interactions.

Issue Difficulty We assess whether ChatGPT is more helpful when assisting with simpler or more complex issues by extracting several features from the issue threads as follows: i) Total number of comments exchanged within the thread (*#Comments*), ii) the total word count across all comments (*Length of Discussion*), iii) time elapsed between the first and last comment (*#Discussion Hours*), iv) the number of distinct developers participating in the discussion (*#Unique Developers*), v) if the issues are resolved and patched, time taken to close the issue (*Resolution time*), vi) the size of the patch based on the number of lines added and removed inside code (*Patch Size*) (Ren et al. 2020; Alencar da Costa et al. 2014; Chen and Jiang 2025; Dhasade et al. 2020). Together, these metrics provide a multifaceted view of issue complexity and effort, helping us assess how task difficulty may influence ChatGPT's helpfulness.

Issue Type To explore the types of issues developers get help with from ChatGPT, we classify issues into thematic categories using unsupervised clustering. We first extract the title and description of each issue and transform them into TF-IDF vectors (Qian et al. 2023). To reduce dimensionality and capture latent semantic patterns, we apply Truncated SVD (LSA) (Al-Msie'deen 2024; França et al. 2025).

Initially, we experimented with HDBSCAN, a density-based clustering algorithm that does not require specifying the number of clusters. While HDBSCAN is generally effective for discovering arbitrarily shaped clusters and filtering noise, we found that it performed poorly on our data due to its sparse and high-dimensional nature. The resulting clusters were overly generalized and noisy, often grouping semantically unrelated issues together or labeling most of the data as outliers. This made it difficult to interpret and assign meaningful categories. Therefore, we opted for K-Means, which is better suited for well-separated, high-dimensional data like TF-IDF representations. K-Means provides clearer and more consistent partitions, allowing us to interpret the resulting clusters more reliably. Using the elbow method (Umargono et al. 2020), we determined that $k = 8$ offered the best trade-off between compactness and separation. In the initial clustering results, the t-SNE (t-distributed Stochastic Neighbor Embedding) projection revealed meaningful groupings but also some notable overlaps between clusters. We observed substantial thematic overlap between clusters 0, 2, 4, and 7, all of which contained variants of improvement-oriented issues (e.g., API feature requests, documentation suggestions, minor usability fixes). To improve interpretability, we merged these four clusters into a single group (cluster 0), resulting in a more coherent and distinct set of themes. The refined clusters, as shown in Fig. 2, consist of a total of 5 clusters, revealing better visual separation and alignment with underlying issue types.

To assign interpretable labels to each cluster, we employ a semi-automated labeling approach. LLMs have shown strong performance in annotating and classifying software engineering data, making them a promising tool for this task (Ahmed et al. 2025; Colavito et al. 2024; Tan et al. 2024). For each cluster, we sample 20 random issues and pass them to GPT-4o with a prompt asking for a short, specific label that captures the type of issue. With cluster sizes ranging from 86 to 223, a random sample of 20 issues yields a margin of error between

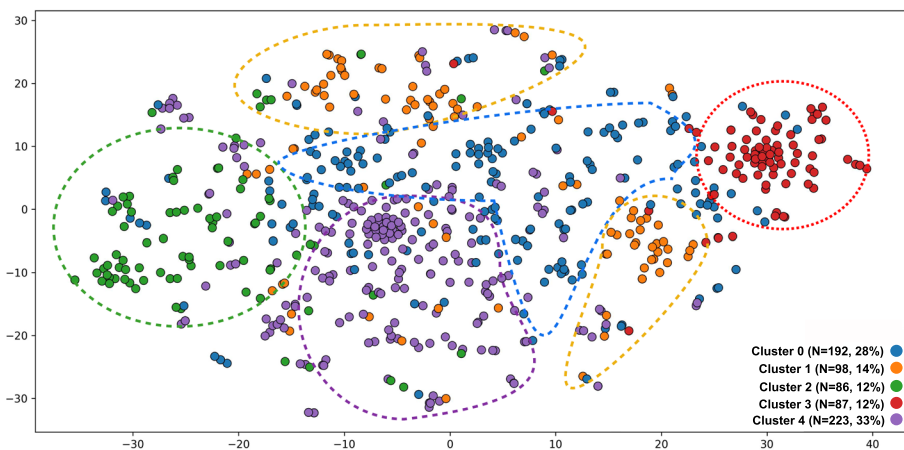


Fig. 2 Final Clusters using K-means

approximately $\pm 19\%$ and $\pm 21\%$ at a 95% confidence level (assuming maximum variability at $p=0.5$). This provides a reasonable balance between coverage and feasibility for high-level thematic labeling. This sampling strategy also follows prior work that found 10–20 examples (or even top keywords) per cluster is sufficient for consistent and interpretable labeling, especially when combined with human validation (Eklund and Forsman 2022; Miller and Alexander 2025; Khandelwal 2025). Therefore, we also manually verified and refined the generated labels by reviewing the cluster contents and ensuring consistency with the developer’s intent.

3.3 RQ3. What are the Common Deficiencies in ChatGPT’s Responses in Unhelpful Issue-Resolution Conversations? How do they Vary Across Different Tasks?

Recent studies have identified several deficiencies in ChatGPT’s responses to software engineering tasks. For instance, they tend to be more verbose and comprehensive than human answers, but often include incorrect information and inconsistencies (Kabir et al. 2024; Li et al. 2023). Other issues are hallucinations, conflicting intents, knowledge gaps, and context deviations (Liu et al. 2024). Building on these prior works, we examine the issue thread discussions of unhelpful developer-ChatGPT conversations, looking for reasons why developers found the conversations unhelpful. Specifically, we qualitatively analyze the negative signals in the issue threads that follow shared ChatGPT conversation links. Comments such as “*ChatGPT is wrong*” (GitHubIssue 2023c) or “*not sure how trustworthy that info is*” (GitHub 2023z) are examples of such indicators in our dataset. Similar to RQ1, to derive a taxonomy of deficiencies, we applied iterative open coding on a random sample of 100 unhelpful instances. The same two annotators independently reviewed each data point and assigned codes based on observed issues in the ChatGPT responses. After each round, they discussed disagreements, refined the code definitions, and repeated the process. This continued until they reached a stable set of five deficiency categories and achieved a Cohen’s Kappa inter-rater agreement of 0.87, indicating strong reliability in the coding process (McHugh 2012). To measure how the deficiencies vary across tasks, we use the taxonomy used in Table 3 and calculate the frequency of deficiencies in each category.

4 Results

4.1 RQ1. How Helpful are the Conversations with ChatGPT in Successfully Resolving Issues?

We identified nine types of SE tasks for which developers sought help from ChatGPT (see Table 3). The most frequent tasks include *Code Generation and Implementation* (196 instances), *Tool/Library/API Recommendation* (182 instances), and *Bug Identification and Fixing* (120 instances). Given that issue threads typically revolve around code and debugging, and that LLMs are known to excel in these tasks (Hou et al. 2024), it is expected that such tasks would be more frequent in our dataset. The least observed tasks were *Test Generation* and *Code Comment Generation*, occurring only in 5 and 4 instances, respectively.

Only 428 out of 686 developer-ChatGPT conversations (62%) were found to be helpful to developers, while 258 were not. This is unexpected given that shared links imply the poster believed the links would be beneficial. The helpful indicators and their distribu-

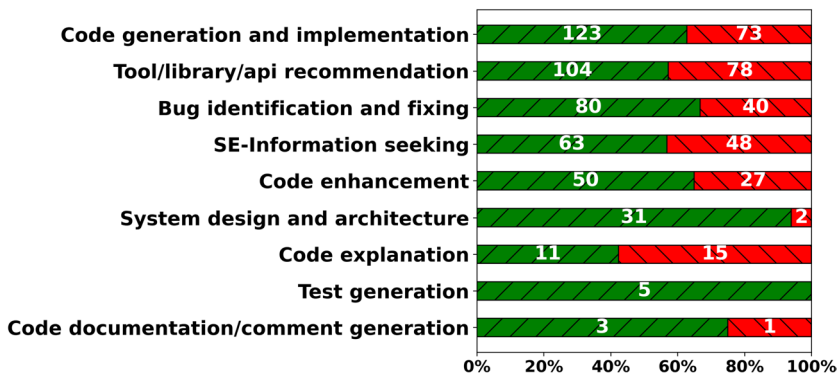


Fig. 3 Number and Percentage of Helpful and Unhelpful Conversations for Each Task. **Green**Accounts for Helpful and **Red**Accounts for Unhelpful

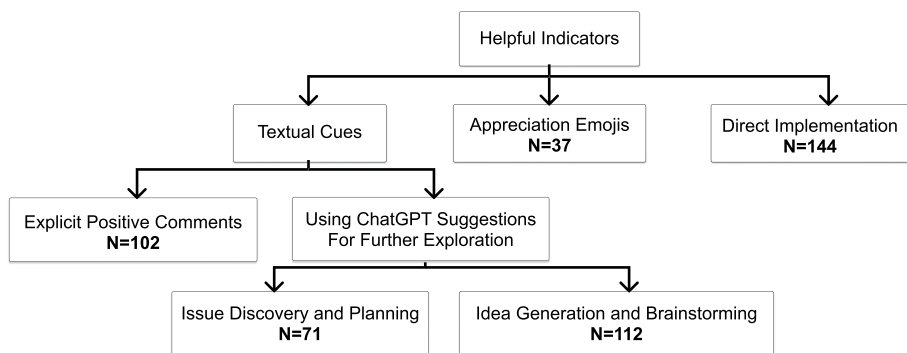


Fig. 4 Helpful Indicators for Developer-ChatGPT Conversations and their Distribution

tion are shown in Fig. 4. The most common indicators include: (1) Direct Implementation ($n = 144$), and (2) Textual Clues, a broader category that includes *Idea Generation and Brainstorming* ($n = 112$) and *Explicit Positive Comments* ($n = 102$). In the case of 103 code-related conversations with ChatGPT, where they lacked clear indicators to determine whether the conversation was beneficial to the issue thread, after running NiCad, only in two instances significant similarities (88% and 100%) were found between the ChatGPT-provided code and the commits.

The perceived efficacy of ChatGPT conversations (percentage of helpful vs. unhelpful) across different issue resolution tasks is shown in Fig. 3. ChatGPT was most effective in addressing *Code Generation and Implementation*, with 123 conversations deemed helpful (62%) and 73 unhelpful. The next most effective tasks were *Tool/Library/API Recommendation* (57% helpful) and *Bug Identification and Fixing* (66% helpful). Developers appreciated the depth of insight provided by ChatGPT, with one stating, “Wow. That is scary and also very informative” (GitHub 2023f) after using ChatGPT to debug DNS null entries in their application. They also praised the adaptability of ChatGPT, with one noting “[ChatGPT] do fairly well going from one syntax to the other” (GitHub 2023g) after using

ChatGPT to translate an Apache config to Nginx. The diagnostic capabilities of ChatGPT in detecting errors were also noticed by developers, with one saying “*I’m definitely adding ChatGPT to my debug toolbox*” (GitHub 2023h) after resolving a series of errors and successfully getting their site running locally. ChatGPT also performed well on *System Design and Architecture* tasks, with 31 out of 33 conversations (94%) labeled as helpful. While this may seem surprising given the high-level system knowledge typically required for such tasks, our analysis revealed that most queries in this category were exploratory in nature. Developers sought brainstorming support, general design strategies, or best practices for implementing architectural components. In these cases, developers often provided sufficient high-level context, enabling ChatGPT to suggest flexible solution paths and design options. This ideation support was often valued by developers, even if the model lacked access to the full system context. However, we note that the relatively small sample size for this task type (n=33) may also contribute to the high success rate observed.

ChatGPT struggled the most with *Code Explanation*, the only task where the number of unhelpful conversations (n=15) was more than the helpful (n=11). In many unhelpful conversations, code snippets were presented to ChatGPT without sufficient context about the codebase or the issue at hand. This affected the accuracy of the answers provided by ChatGPT. ChatGPT also struggled with *SE-Information Seeking*, where developers used ChatGPT to gather specific information related to an issue. For example, developers asked ChatGPT about WebAssembly’s memory usage in Chrome. However, conflicting information from other sources led them to question ChatGPT’s reliability (GitHub 2023n). Based on such patterns in our dataset, using ChatGPT as a search engine for issue resolution may not be recommended, as the model’s inaccuracies, especially in factual queries, can be difficult to detect. While there are instances where ChatGPT was helpful in this role, the risk of subtle or undetected misinformation makes this usage mode less reliable for issue resolution.

Our results also indicate that *Test Generation* and *Code Documentation* were relatively uncommon tasks for which developers engaged ChatGPT in our dataset. While these areas have been well-studied in the broader research community and supported by various LLM-based tools (Schäfer et al. 2024), we observed few instances of developers using ChatGPT conversationally for these purposes during issue resolution. In the limited cases where ChatGPT was applied to these tasks, developers generally found its responses helpful, suggesting that there may be potential in leveraging ChatGPT for these use cases in practice.

428 out of 686 developer-ChatGPT conversations were found helpful in issue resolution (62%). ChatGPT was most effective in tasks like *Code Generation and Implementation*, *Tool/Library/API Recommendation*, and *Bug Identification and Fixing*. However, it struggled with *Code Explanation* and *SE Information-Seeking*.

4.2 RQ2. What are the Key Characteristics of Issue-Resolution Conversations with ChatGPT? Do they Vary Across Helpful and Unhelpful Conversations?

First, we discuss the differences in Conversational Metrics (Structural, Linguistic, Discourse) between helpful and unhelpful conversations. Next, we compare the Project Metrics (Repository, Developer), and finally, we look into Issue Metrics (Difficulty, Type). Our anal-

ysis identifies features that commonly appear in helpful conversations to help characterize patterns in how successful conversations tend to unfold.

4.2.1 Conversational Metrics

We present the results of *Structural* and *Linguistic* metrics in Table 4. These metrics are calculated separately for the developer prompts and ChatGPT responses, as shown in columns (*Developers*) and (*ChatGPT*), respectively. For each row in this table, we show the relative difference (*Rel. Diff.*) by calculating the mean average of the helpful ($h1$) and unhelpful ($h2$), and then determining the difference as $(h1 - h2)/h1$ between helpful and unhelpful conversations. We bold those that are statistically significant according to the Mann-Whitney U test ($p\text{-value} < 0.05$) (Nachar 2008). The Mann-Whitney U test does not assume normal data distribution and is less affected by outliers and skewed data compared to parametric tests (Hollander et al. 2013), making it suitable for our study. We first discuss the results for the metrics in *developer prompts*. We also present corrected p-values using the Benjamini-Hochberg (BH) correction (Benjamini and Hochberg 1995) as an additional reliability measure, to show how many of the differences remain statistically significant after adjustment. We use the BH correction to control the false discovery rate (FDR), reducing the risk of false positives among significant results. With many metrics analyzed, this helps ensure the validity of our findings despite the increased likelihood of false positives from multiple comparisons. In multiple tests, the probability of falsely rejecting at least one null hypothesis (Type I error) increases. This correction adjusts p-values to account for the number of tests, reducing this risk. Unlike stricter corrections like Bonferroni (Armstrong 2014), which control the family-wise error rate and can be overly conservative, the BH correction controls the FDR. This allows for more discoveries while maintaining a low proportion of false positives among significant results (Banerjee et al. 2023). Corrected p-values ensure our findings are less likely due to random chance. Of the 17 metrics initially found to be significant, 11 remained statistically significant after adjustment, indicating that these metrics are robust indicators and less likely to be false positives.

Knowledge Seeking/Sharing In developer prompts, the *#Primary Questions* is higher in helpful conversations (0.25), while the *#Knowledge-seeking Questions* is higher in unhelpful ones (-0.02). However, neither of these measures was statistically significant. We observed that unhelpful conversations contain more clarifying questions. For instance, in one conversation, a developer struggled to find instructions for downloading a model from a website, stating: “I’m trying to download this AI from hugging face and I can’t find any explanation online anywhere...” (GitHub 2023q). After four rounds of back-and-forth interaction, looking for more clarification, the developer asked: “do I absolutely NEED to do the virtual env? Can I just start with pip install transformers? normal answer” and followed up with this knowledge-seeking question: “I don’t have pytorch at all just python and pip. how can I install pytorch”. The conversation continued with more questions about verifying the installation. This pattern explains why unhelpful conversations tend to have more knowledge-seeking questions.

Table 4 Relative Differences between Helpful and Unhelpful Developer-ChatGPT Conversations for Conversational Metrics in Developer Prompts and ChatGPT Answers

Metrics	Sub-categories	Developers			ChatGPT		
		Corrected	Rel. Diff.	Range	Corrected	Rel. Diff.	Range
		P-Value		(Min<Mdn<Max)	P-Value		(Min<Mdn<Max)
Structural: Knowledge Seeking/ Sharing	#Primary Question	0.27	0.25	0 < 1 < 68	-	-	-
	#Knowledge-seeking Question	0.68	-0.02	0 < 0 < 13	-	-	-
	Structural: #API Mentions	0.02	-	0 < 0 < 126	0.99	0.14	0 < 2 < 61
	Contextual		0.39**				
	#URLs	0.68	0.16	0 < 0 < 39	0.88	-0.17	0 < 0 < 9
	#Code Snippets	0.02	0.11**	0 < 1 < 130	0.23	0.28*	0 < 10 < 611
	Mean Size Code Snippets	0.02	-	0 < 18.625 < 17713	0.45	0.04	0 < 56 < 6294
	#Code Descriptions	0.05	-0.21*	0 < 0 < 1242	0.31	0.05	0 < 28 < 3917
	#Error Message	0.40	0.07	0 < 0 < 18	-	-	-
	#Software-specific Terms	0.04	-	0 < 7.5 < 463	0.75	0.001	0 < 44 < 501
Linguistic: Psycholinguistics	#Polite Words	0.04	0.73**	0 < 0 < 33.33	0.74	-0.03	0 < 0 < 7.43
	#Positive Tone Words	0.05	0.48*	0 < 0 < 33.33	0.99	0.05	0 < 0 < 2.5
	#Affect Words	0.04	0.32**	0 < 0.545 < 33.33	0.94	0.01	0 < 1.26 < 11.11
	#Causality Words	0.02	-	0 < 2.975 < 30	0.50	-0.06	0 < 3.16 < 9.69
	#Insight Words	0.05	-0.24*	0 < 2.11 < 25.0	0.98	-0.07	0 < 2.09 < 11.54
	#Cognitive Process Words	0.03	-	0 < 12.48 < 37.5	0.99	-0.03	0 < 14.15 < 25.42
	#Cognition Words	0.02	-	0 < 12.94 < 37.5	0.65	-0.03	0 < 14.42 < 25.81
			0.09**				
	Linguistic: #Unique Words	0.06	-0.22*	1 < 36.5 < 2233	0.99	-0.03	1 < 158 < 2016
	Diversity						
Linguistic: Readability	#Unique Info	0.34	-0.11	1 < 11 < 212	0.97	0.03	1 < 6.66 < 64.33
	#Misspellings	0.79	-0.23	0 < 1 < 145	-	-	-
	#Incomplete Sentences	0.06	-0.63*	0 < 0 < 240	-	-	-
	#Text Speaks	0.88	0.50	0 < 0 < 6	0.99	0.11	0 < 0 < 8
	Flesch Reading Ease Score	0.84	0.03	-790 < 68.77 < 121	0.91	0.02	-5 < 56.93 < 121
	SMOG Grade	0.04	0.20**	0 < 7.45 < 59.9	0.89	-0.02	0 < 12 < 22.5
Linguistic: Verbosity	#Words	0.04	-	1 < 50 < 13893	0.99	0.01	1 < 397 < 13374
	#Sentences	0.05	-0.41*	1 < 3 < 699	0.99	-0.01	1 < 23 < 809

*Indicate differences that are statistically significant with ($p < 0.05$). **Indicate differences that are statistically significant with both ($p < 0.05$) and (*corrected* $p < 0.05$) **Positive** Numbers Indicate Higher Frequencies of Metrics, and **Negative** Numbers Indicate Lower Frequencies in Helpful Conversations

Contextual *#Code Snippets* and *#Error Messages* in developer prompts are higher in helpful conversations, however, the *#Mean Size Code Snippets* is larger in unhelpful conversations, with a relative difference of -0.13. Providing very large code snippets is a common pattern observed in unhelpful conversations in our dataset. This suggests that ChatGPT struggles with handling large code snippets. *#API Mentions* and *#Code Descriptions* are statistically higher in unhelpful conversations, where developers often refer to unclear or incorrect code suggestions made by ChatGPT. The *#URLs* in developer prompts is higher in helpful conversations. Providing additional sources of information could be beneficial for issue resolution. For example, in a helpful conversation, a developer provided a link to help ChatGPT understand the context of the provided code: “Consider [URL]: [CODE] We would like to read frames in memory, not save to file” (GitHub 2023m). However, in unhelpful conversations, we observed a few instances where ChatGPT was unfamiliar with the provided links. This highlights a limitation of ChatGPT and other LLMs regarding data training, as their knowledge is capped at a certain date.

Psycholinguistics All measures are statistically significant and different in developer prompts between helpful and unhelpful conversations. The occurrence of words associated with politeness (*#Polite Words*), positive sentiment (*#Positive Tone Words*), and affection (*#Affect Words*) is notably higher in helpful conversations. This suggests that a positive and polite tone in developer prompts is associated with successful issue resolution. In unhelpful conversations, instances of frustration with ChatGPT’s responses were observed, with developers expressing their discontent through messages like “the zip file name is wrong dimwit” (GitHub 2023y). Unhelpful conversations contain more words related to *causality*, *insight*, and *cognitive* process. This suggests that when developers cannot resolve an issue with ChatGPT, they engage in more complex and analytical thinking. They use more causal, insight, and cognitive words to clarify their issues and make their thought process transparent, hoping for better responses from ChatGPT (Pennebaker et al. 2003; Clark and Brennan 1991).

Diversity Unhelpful conversations exhibit a significantly higher number of *#Unique Words* and *#Unique Info*, compared to helpful ones. LLMs have limitations on the amount of context and tokens they can process (Yang et al. 2024). Providing too much information can be overloading to the LLM, causing confusion about where to direct its attention. We noticed that developers asking for assistance with different tasks within the same conversation thread often led to unsuccessful outcomes.

Readability In developer prompts, unhelpful conversations have more *#Misspellings* and *#Incomplete Sentences*. Additionally, both *Flesch Reading Ease Score* and *SMOG Grade* are higher in prompts of helpful conversations. These scores indicate that developers’ prompts in helpful conversations are well-written and easier to comprehend.

Verbosity The *#Words* and *#Sentences* are both significantly higher in prompts sent in unhelpful conversations. These findings align with prior studies (Hao et al. 2024), which indicate that developers often engage in iterative back-and-forth loops with ChatGPT, seeking clarification, follow-ups, and prompt refinements to find answers. These unsuccessful loops are prevalent in unhelpful conversations, leading to the increased number of

Table 5 Sentiment and Prompt Statistics for Helpful vs. Unhelpful Conversations

ChatGPT Conversations	Sentiment			#Words	#Sentences	Median #Prompts	Rounds of Conversation			
	Neg.	Neut.	Pos.				0 to 10	10 to 20	20 to 25	25 >
Helpful (n=428)	70 (16%)	284 (66%)	75 (18%)	61348	1861	2 (93%)	401 (4%)	20 (~1%)	4 (~1%)	3
Unhelpful (n=258)	38 (15%)	192 (74%)	28 (11%)	106476	3558	2 (93%)	241 (4%)	11 (4%)	5 (~1%)	1

words and sentences within these conversations. As shown in Table 5, the distribution of prompt counts across helpful and unhelpful conversations is nearly identical, with 93% of conversations in both groups involving 0-10 prompts. This suggests that the number of prompts alone does not distinguish helpfulness; rather, it is the efficiency of the exchanges that matters. Helpful conversations may include multiple rounds, but they tend to be more concise, enabling developers to iterate toward solutions with fewer words and sentences overall.

Structural and Linguistic metrics in the *ChatGPT responses* indicate only one feature with a statistically significant difference between helpful and unhelpful conversations: *#Code Snippets*, with a relative difference of 0.28. ChatGPT's answers in helpful conversations contained more code snippets, driving effective issue resolution. The other features showed minimal differences, suggesting that ChatGPT maintains a consistent tone and language style across its conversations with developers, in both helpful and unhelpful conversations.

We present the results of *Discourse* metrics in Figs. 5, 6, and 7. In each figure, we plot the average scores for each metric (Topic Coherence, Lexico Semantic Similarity, Linguistic Style Matching). In these figures, A "conversation step" refers to each round of interaction, consisting of one developer prompt and one ChatGPT response, and the shaded areas in the plots represent the standard deviation of the metrics in helpful (blue) and unhelpful (red) conversations throughout the discourse. In Table 5, we provide descriptive statistics on conversation rounds, word and sentence counts, and user sentiment for both helpful and unhelpful conversations. The distributions of conversation rounds are nearly identical across the two groups. For instance, the majority of the conversations occur within 0 to 10 rounds (401 out of 428 helpful conversations, 241 out of 258 conversations).

The sentiment patterns also follow a similar trend, with the majority of prompts being neutral in both helpful and unhelpful conversations. However, helpful conversations exhibit a greater proportion of positive sentiment (18%) compared to unhelpful ones (11%), indicating that more positively framed conversations tend to be perceived as more helpful. Prompting with phrases such as "*I'd love for you to point me to some examples*" or expressions of gratitude are examples of such positive conversations (GitHubIssue 2023a).

Topic Coherence Topic coherence scores are generally higher in helpful conversations compared to unhelpful ones, as shown in Fig. 5. Drops in scores signify topic deviations between consecutive steps (e.g., noticeable drop between steps 8 and 9 in unhelpful conversations). One such instance was where a developer first sought help with a specific error:

Fig. 5 Topic Coherence Measured for Helpful/Unhelpful Conversations

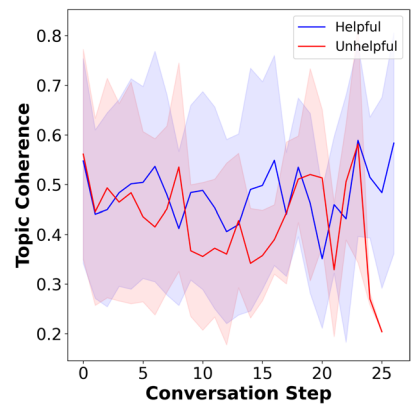


Fig. 6 Lexico-Semantic Similarity Measured for Helpful/Unhelpful Conversations

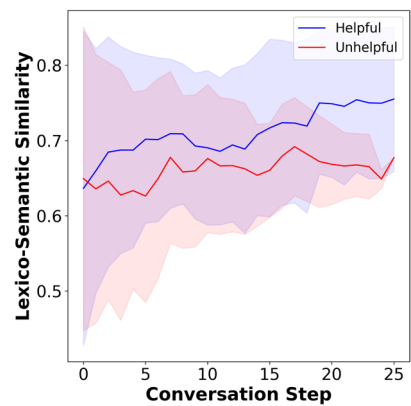
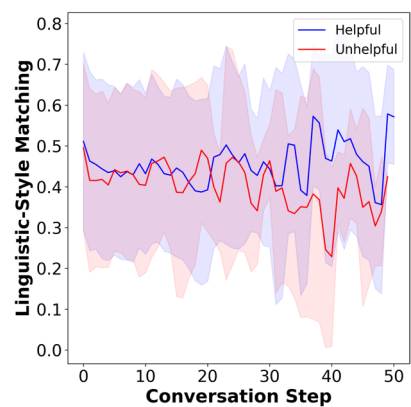


Fig. 7 Linguistic-Style Matching Measured for Helpful/Unhelpful Conversations



“(PY0001): PyScript: Access to local files...[ERROR]. See [URL]...” (GitHub 2023u), to which ChatGPT provided potential solutions. Immediately at the next step, the developer abruptly shifted to a different topic: “how to put python code in website”. These abrupt topic changes were more common in unhelpful conversations.

Lexico-Semantic Similarity As shown in Fig. 6, lexico-semantic similarity increases for both helpful and unhelpful conversations, indicating that developer prompts and ChatGPT responses become more semantically aligned as the conversation progresses. However, helpful conversations consistently show higher similarity. Our analysis reveals that this score decreases when developers include error messages and code snippets without accompanying explanations or specific requests, a pattern frequently observed in unhelpful conversations. Score drops also occur due to misalignment in understanding the developer's request. For example, in an unhelpful conversation (GitHub 2023t), the developer initially asked: *"create load generator in go"*, to which ChatGPT provided an unrelated answer that did not meet the developer's needs. The developer then clarified with: *"I mean CPU load"*, highlighting the misalignment. These kinds of misunderstandings also contribute to score drops, which are more common in unhelpful conversations.

Linguistic-Style Matching (LSM) The linguistic styles of developers and ChatGPT are generally similar in both helpful and unhelpful conversations, as shown in Fig. 7. However, this trend shifts at step 32 with helpful conversations showing greater linguistic alignment. Helpful conversations generally involve developers making an effort to structure their prompts clearly, facilitating ChatGPT's understanding and providing better assistance. For instance, in a high-scoring LSM conversation, a developer tries to adjust an application feature with ChatGPT's help. Throughout the interaction, the developer tries the scripts provided by ChatGPT and gives clear feedback on the results (GitHub 2023r): *"It seems there was an error in the script, and it appears that the [CODE] library does not have a constant named [CODE]"*, or *"this is very close. I am wondering why it is moving the left keyboard arrow"*. This clear communication ultimately leads to resolving the issues with the script.

4.2.2 Project Metrics

We present the results in Table 6. For each row in this table, we show the relative difference (Rel. Diff.) between the projects involved in helpful and unhelpful conversations, bolding those that are statistically significant according to the Mann-Whitney U test (p -value < 0.05) (Nachar 2008). We also add Benjamini-Hochberg corrected values in the table.

Table 6 Relative Differences Between Helpful and Unhelpful Developer-ChatGPT Conversations for Project Metrics

Project Metric	Categories	P-value	Corrected P-Value	Rel. Diff.	Range (Min<Median<Max)
Repository Parameters	#Stars	0.73	0.91	0.27	0 < 23 < 116, 455
	#Contributors	0.39	0.65	-0.01	1 < 5 < 30
	#Forks	0.83	0.83	-0.01	0 < 6 < 35, 443
	#Files	0.0002	0.001	0.40**	0 < 85 < 48, 122
	#Lines of Code	0.0008	0.002	0.72**	0 < 12, 936 < 103, 840, 135
Developer Experience	#Public Repositories	0.18	0.18	0.18	0 < 42 < 8, 728
	#Followers	0.10	0.13	0.24	0 < 27 < 42, 042
	Account Age	0.05	0.10	0.10	0 < 9 < 16
	#Contributions	0.03	0.12	0.05*	0 < 20 < 30

*Indicate differences that are statistically significant with ($p < 0.05$). **Indicate differences that are statistically significant with both ($p < 0.05$) and (corrected $p < 0.05$) **Positive** Numbers Indicate Higher Frequencies of Metrics, and **Negative** Numbers Indicate Lower Frequencies in Helpful Conversations

Repository Parameters The *#Files* and *#Lines of Code* are both statistically significant and higher in helpful conversations, indicating that projects with extensive codebases tend to have more effective interactions with ChatGPT. Additionally, *#Stars* is also higher in helpful conversations, suggesting that popular projects benefit more from ChatGPT's assistance. Conversely, projects associated with unhelpful conversations have slightly more contributors and forks, although they are not statistically significant.

Developer Experience All measures in this category are higher in helpful conversations, with *#Contributions* being statistically significant. This suggests that effectively leveraging ChatGPT for issue resolution is not solely about access to the tool; it also depends on the developer's experience and familiarity with software development workflows. Conversations between experienced developers and ChatGPT are more likely to be helpful compared to those involving less experienced developers. This is especially important in the context of the new trend of *vibe coding*, where less experienced developers may rely heavily on LLMs without fully understanding the system. Our findings suggest that such reliance, without sufficient experience, increases the risk of unhelpful or misleading outcomes. Experienced developers tend to provide more context, clearer questions, and more relevant information, which likely leads to better responses from ChatGPT. Additionally, experienced developers are better at interpreting ChatGPT's responses and providing follow-up queries or corrections, further enhancing the conversation's productivity and helpfulness. This highlights the critical role of experience in effectively using ChatGPT for problem-solving and issue resolution.

4.2.3 Issue Metrics

We present the results for Issue Difficulty in Table 7. For each row, the table reports the relative difference (Rel. Diff.) between issue threads associated with helpful and unhelpful conversations. Rows that are statistically significant based on the Mann-Whitney U test ($p\text{-value} < 0.05$) are shown in bold. We also include Benjamini-Hochberg corrected $p\text{-values}$ to account for multiple comparisons and reduce the likelihood of false positives.

Table 7 Relative Differences Between Helpful and Unhelpful Developer-ChatGPT Conversations for Issue Difficulty

Issue Metric	Categories	P-value	Corrected P-Value	Rel. Diff.	Range (Min<Median<Max)
Issue Difficulty	#Comments	0.82	0.96	-0.16	0<3<30
	Length of Discussion	0.54	0.76	-0.42	0 < 77 < 8378
	#Discussion Hours	0.93	0.93	0.002	0 < 238 < 68591
	#Unique Developers	0.001	0.002	-0.11**	1 < 2 < 24
	Resolution time	0.001	0.002	-0.002**	0 < 294 < 68591
	Patch Size	0.001	0.002	0.70**	0 < 0 < 26718

*Indicate differences that are statistically significant with ($p < 0.05$). **Indicate differences that are statistically significant with both ($p < 0.05$) and (*corrected* $p < 0.05$) **Positive** Numbers Indicate Higher Frequencies of Difficult Issues, and **Negative** Numbers Indicate Lower Frequencies in Helpful Conversations

Table 8 Identified Issue Types for Helpful and Unhelpful Conversations and their Frequencies

Issue Type	Total Conversations	Helpful Conversations	Unhelpful Conversations
Debugging and Testing Issues	223	134 (60%)	89 (40%)
API and Library Feature Requests	192	125 (65%)	67 (35%)
Installation and Configuration Issues	98	63 (64%)	35 (36%)
Performance Optimization and Code Refactoring	87	48 (55%)	39 (45%)
Compatibility and Compilation Errors	86	58 (67%)	28 (33%)

Table 8 presents the distribution of issue types identified for each conversation. We identified five distinct issue types across the dataset:

API/Library Feature Requests, such as issues requesting new functionality, methods, or improvements to existing APIs and libraries.

Installation and Configuration Issues, problems related to project setup, environment configuration, dependency management, and onboarding.

Compatibility and Compilation Errors, characterized by errors stemming from version mismatches, platform incompatibility, or build failures.

Performance Optimization and Code Refactoring, involving reports focused on slow runtime, high memory usage, or suggestions for improving code efficiency and structure.

Debugging and Testing Issues, including bugs encountered during development, test failures, and issues involving mocking, test coverage, or flaky tests.

The table reports the frequency of each issue type and its distribution across helpful and unhelpful conversations.

These issue types also align closely with the tasks in the conversations we identified in RQ1. For example, *Debugging and Testing Issues* often involve tasks like Bug Identification and Fixing, while *API/Library Feature Requests* correspond to Tool/ Library/API Recommendation. Similarly, *Compatibility and Compilation Errors* and *Installation and Configuration Issues* often prompt developers to seek Code Generation or implementation help. This overlap reinforces that the issue types reflect the nature of the developer-ChatGPT conversations in our dataset.

Issue Difficulty Unhelpful conversations are more frequently associated with issue threads that show higher engagement from the developers, reflected in a greater number of comments (*#Comments*), longer discussions (*Length of Discussion*), and more participating developers (*#Unique Developers*). Among these, only *#Unique Developers* is statistically significant. Additionally, unhelpful conversations are associated with significantly longer resolution times (*Resolution Time*). These findings suggest that unhelpful conversations are more likely to occur in complex, collaborative, and time-consuming issues. Coordination across multiple contributors reflects more complex or ambiguous issues where ChatGPT struggles to provide actionable help. On the other hand, the only metric significantly higher in helpful conversations is *Patch Size*, indicating that when ChatGPT is helpful, it contributes to larger and more substantial code changes by helping developers more confidently make broader updates or refactorings.

Issue Type ChatGPT is most helpful in resolving *Compatibility and Compilation Errors* (67% helpful), *API and Library Feature Requests* (65% helpful), and *Installation and Configuration Issues* (64% helpful). These types of issues often involve focused, well-scoped technical questions related to known APIs, dependencies, or environment setup, contexts where ChatGPT can provide actionable suggestions based on its pretraining. In contrast, ChatGPT is relatively less helpful for *Performance Optimization and Code Refactoring* (45% unhelpful) and especially for complex *Debugging and Testing Issues* (40% unhelpful), which may involve subtle logic errors or incomplete bug descriptions. These tasks often require a deeper understanding of the project's runtime behavior, internal state, or debugging context, information that is often missing or underspecified in prompts. These findings reinforce a key limitation of general-purpose LLMs: their effectiveness decreases when issue resolution depends on nuanced, dynamic, or implicit project-specific information that is not directly provided in the conversation.

Conversational Metrics: Developer prompts in helpful conversations are more readable (SMOG Grade), polite (#Polite Words), and coherent (Topic Coherence). In contrast, unhelpful conversations are more verbose (#Words), overloaded with too much information (#Unique Info) and contain longer code snippets (Mean Size Code Snippets).

Project Metrics: Larger projects (#Files) use ChatGPT more effectively for issue resolution, and experienced developers (#Contributions) leverage ChatGPT's capabilities more efficiently.

Issue Metrics: ChatGPT is less helpful for complex, time-consuming issues (*Resolution Time*) that involve high developer collaboration (#Comments, *Length of Discussion*, #Unique Developers). It performs best on well-defined technical issues like compilation errors and feature requests, but struggles with context-heavy debugging and refactoring issues that require deeper, project-specific understanding.

4.3 RQ3. What are the Common Deficiencies in ChatGPT's Responses in Unhelpful Issue-Resolution Conversations? How do they Vary Across Different Tasks?

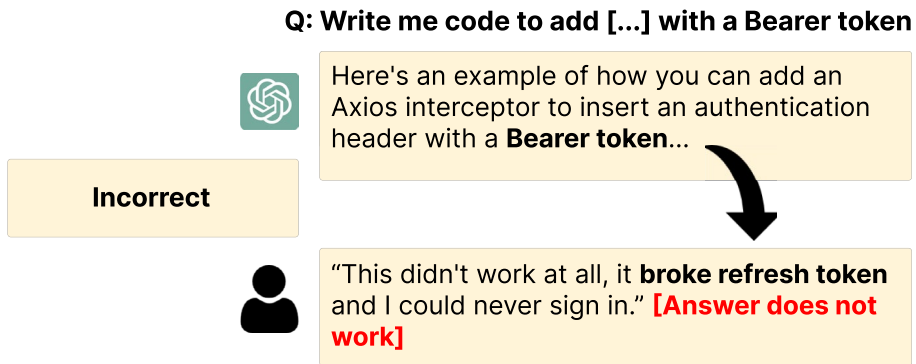
We analyze 258 unhelpful developer-ChatGPT conversations to identify deficiencies in ChatGPT's responses. We categorize these deficiencies into five categories: incorrect information (*Correctness*), insufficient or incomplete information (*Comprehensiveness*), unclear information leading to confusion (*Clarity*), outdated or irrelevant information (*Relevance*), and fabricated or nonsensical responses (*Hallucination*).

ChatGPT's answers were identified as *Incorrect* in 54 cases, *Not Comprehensive* in 39 cases, *Unclear* in 14 cases, *Irrelevant* in 19 cases, and *Hallucination* in 11 cases. The remaining 121 conversations are unresolved issue threads without clear indications of any specific deficiencies. The distribution of deficiencies across different tasks is shown in Table 9.

Incorrect answers are factually wrong or inaccurate responses, as reported by developers in the issue threads after testing or examining the provided solutions. For example, a developer mentioned, "I tried again [URL] and the regex it gave me is failing..." (GitHub 2023i). We also identified several instances where developers flagged the code provided by ChatGPT as bad practice, leading them to avoid using it for issue resolution (Fig. 8).

Table 9 Number of Deficiencies in Unhelpful Conversations by Task

	Incorrect	Not Comprehensive	Unclear	Irrelevant	Hallucination
Code Generation and Implementation	12	7	0	6	1
Tool/Library/API Recommendation	23	8	5	8	5
Bug Identification and Fixing	13	3	1	3	1
SE Information-Seeking	0	15	5	4	4
Code Enhancement	5	3	2	2	0
System Design and Architecture	0	1	0	0	0
Code Explanation	3	5	1	1	0
Test Generation	0	0	0	0	0
Code Documentation/Comment Generation	0	0	0	0	0

**Fig. 8** Example of Incorrect Deficiencies in ChatGPT's Answers

The highest number of incorrect answers were noticed in the three most frequent tasks of *Tool/Library/API Recommendation* (n=23), *Bug Identification and Fixing* (n=13), and *Code Generation and Implementation* (n=12).

Not comprehensive answers are responses that were found to be insufficient or incomplete for the developers. For instance, one developer mentioned, “Hm, not too bad, but also still very generic...” (GitHub 2023j). Another example of this is shown in Fig. 9. Most instances of not comprehensive answers were observed in the tasks of *SE Information-Seeking* (n=15), *Tool/Library/API Recommendation* (n=8), and *Code Generation and Implementation* (n=7), where the answers and information given by ChatGPT were too generic and incomplete to help resolve the issue.

Unclear answers are responses where the information by ChatGPT is vague and causes confusion among the developers (Fig. 10). The most frequent tasks with this deficiency are *SE Information-Seeking* (n=5), *Tool/Library/API Recommendation* (n=5), and *Code Enhancement* (n=2). Across different tasks, we observed a recurring issue with the vagueness of ChatGPT's answers. For example, during a feature enhancement discussion, a developer noted, “[ChatGPT] got me absolute vagueries about the specifics” (GitHubIssue 2023b).

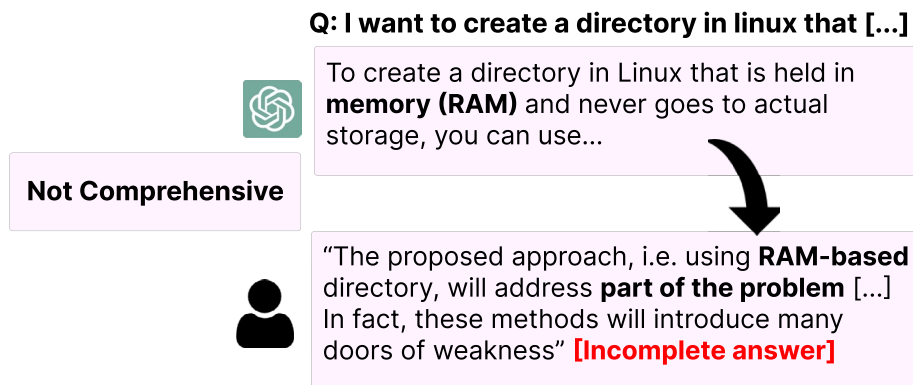


Fig. 9 Example of Not Comprehensive Deficiencies in ChatGPT's Answers

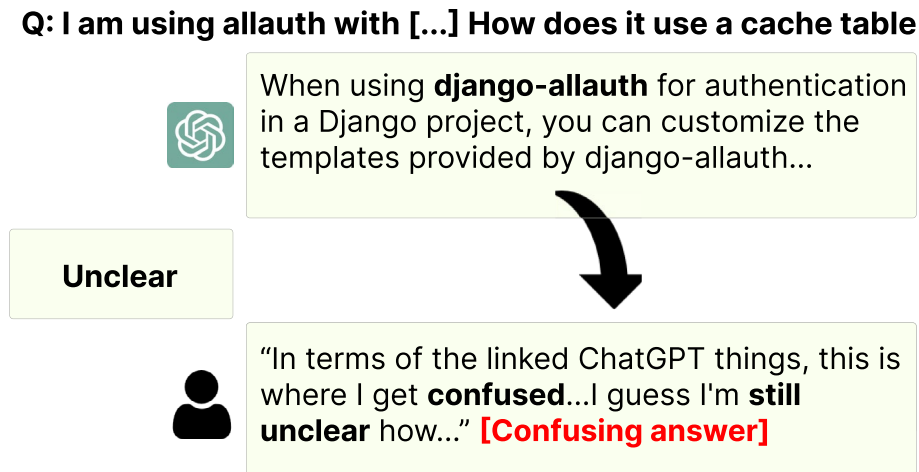


Fig. 10 Example of Unclear Deficiencies in ChatGPT's Answers

Irrelevant answers are responses that were outdated or unrelated to the context of the issue (Fig. 11). These deficiencies were mostly seen in *Tool/Library/API Recommendation* (n=8), *Code Generation and Implementation* (n=6), and *SE Information-Seeking* (n=4). In tool recommendation and code generation tasks, irrelevance is often related to the answer's inconsistency with the problem at hand. For example, a developer mentioned, "*that is an entirely unrelated route for users to authenticate*" (GitHub 2023p). In information-seeking tasks, ChatGPT often provided outdated or unrelated information. For instance, a developer noted, "*[Library] is mentioned in the documentation somewhere but noticed that is an archived repo so not maintained right?*" (GitHub 2023s).

As for *Hallucination*, the most frequent patterns involved the fabrication of non-existent methods, API calls within various libraries, and non-existent facts (Fig. 12). Hallucinations were most commonly observed in *Tool/Library/API Recommendation* (n=5) and *SE Information-Seeking* (n=4). For example, a developer trying to use a library based on ChatGPT's

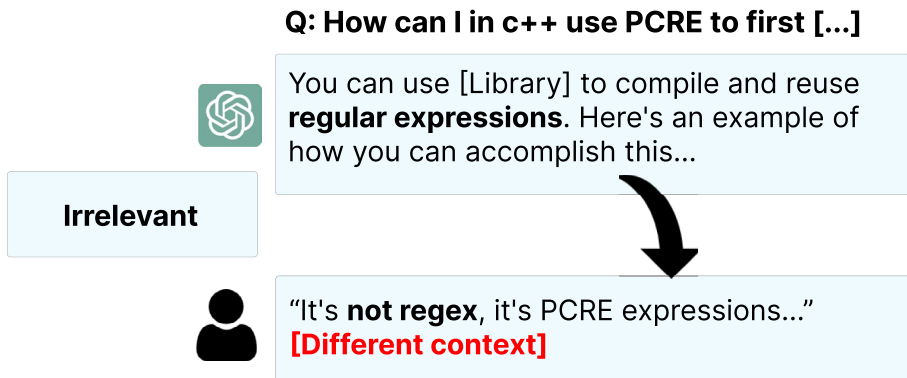


Fig. 11 Example of Irrelevant Deficiencies in ChatGPT's Answers

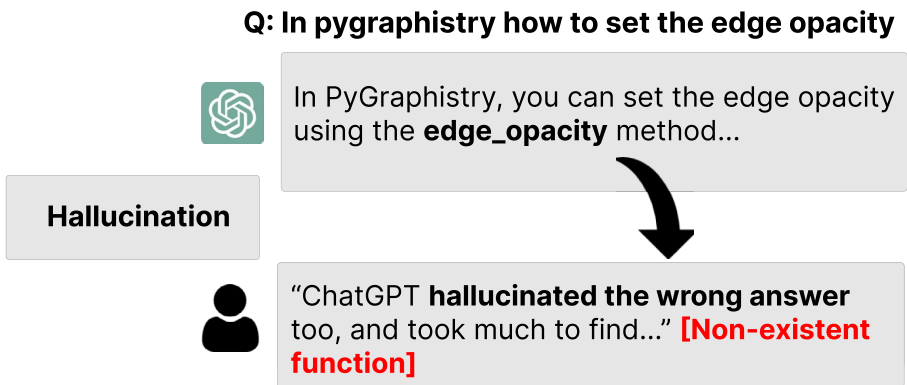


Fig. 12 Example of Hallucination Deficiencies in ChatGPT's Answers.

suggestion noted, "*ChatGPT gave a decent suggestion of what this might look like, but neither the source code nor the documentation has any hints that this exists*" (GitHub 2023k). Another instance was found in an issue thread where developers used ChatGPT as a search engine for their project, leading to a conclusion about the hallucinations in the answers: "*...ChatGPT hallucinates, so is dead in the water as a reasonable search engine...*" (GitHub 2023l).

The two most common deficiencies in ChatGPT's responses were information *incorrectness* and lack of *comprehensiveness*. Among other tasks, *Tool/Library/API Recommendations* contained the highest number of deficiencies.

5 Discussion and Actionable Insights

In this section, we present actionable insights for practitioners and researchers for issue resolution, focusing on: a) interaction strategies with conversational LLMs, b) tools to optimize prompt design, and c) considerations for fine-tuning conversational LLMs.

Interaction Strategies Our findings indicate that specific and concise prompts are key to effective issue resolution. Helpful conversations exhibit higher scores for contextual metrics (e.g., *#Code Snippets*, *#Error Messages*, *#URLs*) and readability metrics (e.g., *SMOG Grade*), while avoiding verbosity with fewer *#Words* and *#Sentences*. These observations suggest that developers should prioritize providing detailed yet focused descriptions of issues, specifying key information, and avoiding broad or overly general queries. One notable pattern in unhelpful conversations is the presence of lengthy prompts with multiple questions, large code snippets, and excessive or unfocused error messages. When addressing complex issues, developers can improve conversation outcomes by dividing the problem into discrete parts and tackling each individually.

We suggest avoiding frequent topic changes within the same conversation, as our discourse metrics (e.g., *Topic Coherence*) show that unhelpful interactions often shift between unrelated topics, leading to disjointed responses. In addition, providing feedback during the conversation enhances alignment with the developer's needs. In helpful conversations, developers, especially those with more experience, often offered specific, targeted feedback on responses, indicating what was useful and highlighting areas for improvement. They were more effective at steering the conversation, clarifying ambiguous or incorrect responses, and breaking down complex tasks into smaller, manageable queries. This approach keeps ChatGPT focused and helps guide the conversation back on track if responses start to deviate.

Tools for Prompt Optimization Our findings can inform development of tools to optimize conversational LLM prompts for effective issue resolution. Our conversational metrics-structural, linguistic, and discourse-can help flag and improve inefficient prompts (e.g., automatically rephrasing issue descriptions to improve *readability*, urging developers to *include code snippets*). Tools providing real-time feedback and prompt optimization before developers submit queries to ChatGPT could significantly enhance response quality.

Specialized Conversational LLMs Our analysis shows that ChatGPT is more effective in tasks like *Code Generation and Implementation*, *Tool/Library/API Recommendation*, and *Bug Identification and Fixing*, but struggles with *Code Explanation* and *SE Information-Seeking* related to issue resolution. Common issues, such as incorrect, irrelevant, incomplete, and unclear responses, were particularly prevalent in the *Tool/Library/API Recommendation* and *SE Information-Seeking* tasks. Our analysis also revealed instances of code-breaking suggestions and poor coding practices in ChatGPT responses, which further highlights the risks of using general-purpose conversational LLMs for SE-specific tasks. These findings highlight areas where ChatGPT performs well and where it fails to meet expectations. With these insights, we can focus on fine-tuning conversational LLMs to improve their performance in specific tasks, with the goal of developing specialized conversational LLMs for issue resolution.

Retrieval-Augmented Generation and Agentic Workflows Many unhelpful conversations stemmed from a lack of project-specific knowledge or incomplete context. Our findings highlight how this gap strongly influences conversation outcomes, as reflected in contextual metrics such as the inclusion of *#Code Snippets*, *#Error Messages*, and *#URLs*. We also found that ChatGPT was less effective on issue types such as *Debugging and Testing*

Issues and Performance Optimization and Code Refactoring Issues, which often require a deeper understanding of project internals, infrastructure, or system configurations. In such cases, Retrieval-Augmented Generation (RAG) can be used to ground ChatGPT's responses in relevant documentation, codebases, or project artifacts. Automatically integrating retrieval pipelines into developer tools could enable ChatGPT to produce more context-aware responses without the need for full fine-tuning. By grounding responses in external sources of truth, RAG also helps reduce common deficiencies we identified, such as hallucinated content, factual inaccuracies, and incomplete or unclear suggestions (Shuster et al. 2021; Ayala and Bechard 2024). In addition, issue resolution often requires handling multiple interconnected steps, something conversational LLMs struggle with in a single exchange. Our findings on issue difficulty show that ChatGPT is less effective on complex, time-consuming issues involving higher collaboration and longer resolution times. Agentic workflows offer a promising alternative by allowing the model to break down complex tasks into smaller subtasks, solve them step by step, and reflect on intermediate results along the way. Future systems that incorporate these agentic patterns may improve reliability and effectiveness for tasks such as test generation and multi-stage debugging. Together, RAG and agentic workflows represent practical and scalable ways to extend the capabilities of general-purpose conversational LLMs for real-world software engineering scenarios.

6 Threats to Validity

Construct Validity As with any study involving manual analysis, the results of this study are subjective to human judgment. To limit this threat, we ensured that the authors who analyzed the data had considerable experience in programming (3+ years) and qualitative analysis, and followed a consistent coding procedure that was piloted in advance. We conducted multiple rounds of coding and discussions to ensure consistency and resolve disagreements. Our average Cohen's Kappa agreement was 0.88, indicating sufficient reliability.

Internal Validity The metrics that we used to measure conversational and project differences may have inherent biases or inaccuracies. For instance, how easy a conversation is to read i.e. *readability* may differ from one developer to another, and the *size of the project* might not directly affect the effectiveness of the usage of ChatGPT. To mitigate these threats, we used the Mann-Whitney test to statistically validate the differences between helpful and unhelpful conversations. By having multiple metrics for each category, conversational and project, we also ensured that our results were not biased by a single metric. Other potential threats include evaluation bias or script errors. To mitigate these, we performed sanity tests for all automated metrics. Additionally, while our multi-indicator approach to labeling helpfulness (including clone detection and manual inspection) helps reduce misclassification, there may still be rare edge cases where developer intent is ambiguous or misleading (e.g., expressions of gratitude despite unhelpful responses).

External Validity Our findings are based on a dataset where developers shared links to their ChatGPT conversations within GitHub issue threads. Therefore, our results may not generalize to other ChatGPT conversations that developers choose not to share on GitHub. Our observations also might not generalize to developer conversations with other LLMs such as

Llama and Copilot. To mitigate these threats, we selected ChatGPT due to its widespread popularity and extensive use among developers. In addition, our extensive dataset, collected from 655 projects comprising 48 programming languages, includes a diverse range of developer-ChatGPT interactions. To further assess coverage, we conducted a follow-up crawl for the subsequent 13-month period (May 2024 - May 2025) since the start of our study. We found only 125 additional conversations, compared to the 876 in our original dataset. With 876 out of a known 1,001 total conversations, our sample achieves a $\pm 1.2\%$ margin of error at the 95% confidence level (assuming maximum variability), indicating that our dataset remains highly representative of the available population during the extended timeframe (Cochran 1977).

7 Related Work

LLMs for Bug and Issue Resolution LLMs, especially conversational LLMs, are increasingly used in bug localization, bug resolution, and issue tracking (Hou et al. 2024; Wu et al. 2023; Tang et al. 2023; Ehsani et al. 2025a). Previous studies have investigated the effectiveness of LLMs in this context and found that developers often interact with ChatGPT to describe bug symptoms, provide contextual information, and seek guidance on potential solutions (Das et al. 2024). Tao et al. (2024) developed an LLM-powered multi-agent framework to solve GitHub issue threads, achieving a resolution rate of 13.94%. Q&A platforms like Stack Overflow have been crucial for assisting developers in issue resolution, but their traffic declined with the rise of conversational LLMs (Silva et al. 2024). Several studies compared Q&A forum answers to LLM responses, and found that LLMs struggle with questions related to unpopular frameworks and libraries (Silva et al. 2024). For security questions, ChatGPT responses were correct only 56% of the time (Delile et al. 2023). Despite these concerns, users prefer ChatGPT for their well-articulated language (Kabir et al. 2024). ChatGPT answers were consistently found to be of lower quality than Stack Overflow, often lacking relevance and information timeliness (Xu et al. 2023). Human expertise to solve specific exceptions makes Stack Overflow better in debugging tasks (Liu et al. 2023). Overall, these studies highlight the evolving role of LLMs in issue resolution. We contribute to the existing knowledge by identifying specific SE tasks that LLMs effectively support for issue resolution and evaluating their helpfulness for each task.

Analysis of Developer-LLM Conversations Understanding how software developers interact with ChatGPT has been a focus in recent works. The most common inquiries to ChatGPT are related to code generation, and obtaining answers to conceptual and how-to questions (Hao et al. 2024), and the most commonly identified topics posed by developers are advanced programming guidance, information-seeking about frameworks, and obtaining high-level design recommendations (Mohamed et al. 2024; Ertugrul Sagdic 2024). Developers often engage in multi-turn conversations to improve response quality by asking follow-up questions or refining prompts (Hao et al. 2024), LLM-generated code is primarily used for demonstrating high-level concepts or providing examples in documentation. Most of the conversations are about requesting improvements and more description inside generated codes (Jin et al. 2024). Mondal et al. identified 11 gaps that make ChatGPT conversations lengthy, with missing specifications and requests for additional functionality being the most frequent reasons (Mondal et al. 2024). Champa et al. (2024) found that developers

mostly seek assistance from ChatGPT with Python code for quality management and issue resolution tasks, and developer-ChatGPT conversations are the most efficient for software development management, optimization, and new feature implementation (Champa et al. 2024). Recent work has further revealed that developers primarily use ChatGPT for ideation, while direct reliance on its generated code to resolve issues is minimal. The study also identified areas like refactoring and data analytics where ChatGPT excels, and debugging and automation tasks where dissatisfaction is higher (Das et al. 2025). Our paper complements existing work by providing insight into the dynamics and characteristics that make developer-ChatGPT conversations successful in the context of issue resolution.

8 Conclusion and Future Work

In this study, we explored the usage and effectiveness of conversational LLMs for issue resolution within GitHub's issue threads. We found that only 62% of the developer-ChatGPT conversations helped in resolving issues across tasks such as *Code Generation and Implementation*, *Tool/Library/API Recommendation*, and *Bug Identification and Fixing*. We identified that helpful and unhelpful conversations have key conversational differences in their structure, linguistic patterns, and discourse. Project metrics show that larger and more popular projects, as well as more experienced developers, use ChatGPT more effectively for issue resolution. Our issue-level analysis further reveals that ChatGPT is more effective on tasks such as feature requests and compilation errors, but tends to struggle on complex, time-consuming problems involving multiple developers, such as debugging and project refactoring. Our findings provide actionable insights for practitioners and researchers, including strategies for effective interactions, guidance for developing tools to optimize prompts, and recommendations for fine-tuning LLMs for issue resolution tasks.

In the future, we plan to experiment with fine-tuned LLMs and develop a system that streamlines interactions with conversational LLMs by integrating identified metrics into developer prompts. We will conduct experiments to evaluate whether these enhanced interactions lead to better issue resolution outcomes and a more positive user experience.

Author Contributions All authors contributed to the study conception and design. Material preparation, data collection and analysis were performed by Ramtin Ehsani. The first draft of the manuscript was written by Ramtin Ehsani and all authors commented on previous versions of the manuscript. All authors read and approved the final manuscript.

Funding No funding was received for conducting this study.

Data Availability The replication package of our study, including the datasets, code, and analysis instructions are available at: <https://figshare.com/s/2a7284f4c593888a35cc>

Declarations

Competing Interests The authors declare that they have no conflict of interest.

Ethics Declaration Not applicable.

Informed consent Not applicable.

Open Access This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit <http://creativecommons.org/licenses/by/4.0/>.

References

- Ahmed T, Devanbu P, Treude C, Pradel M (2025) Can llms replace manual annotation of software engineering artifacts? [arXiv:2408.05534](https://arxiv.org/abs/2408.05534)
- Al-Msie'deen R (2024) Bushradbr: An automatic approach to retrieving duplicate bug reports. *Int J Comput Digi Syst* 15(1):221–238. <https://doi.org/10.12785/ijcds/150118>
- Armstrong RA (2014) When to use the Bonferroni correction. *Ophthalmic Physiol Opt J Br Coll Ophthalmic Opticians (Optometrists)* 34(5):502–508. <https://doi.org/10.1111/opo.12131>
- Ayala O, Bechard P (2024) Reducing hallucination in structured outputs via retrieval-augmented generation. In: Yang Y, Davani A, Sil A, Kumar A (eds) *Proceedings of the 2024 conference of the North American Chapter of the association for computational linguistics: Human language technologies (Volume 6: Industry Track)*, association for computational linguistics, Mexico City, Mexico, pp 228–238. <https://doi.org/10.18653/v1/2024.naacl-industry.19>
- Banerjee S, Sussman M, Lian Y (2023) Dimensional analysis in error reduction for prediction of nucleate boiling heat flux by artificial neural networks for limited dataset. *ASME J Heat Mass Transfer* 145(6):061602
- Barrus T (2024) pypspellchecker: Pure python spell checker based on work by Peter Norvig
- Benjamini Y, Hochberg Y (1995) Controlling the false discovery rate: A practical and powerful approach to multiple testing. *J Roy Stat Soc: Ser B (Methodol)* 57(1):289–300. <https://doi.org/10.1111/j.2517-6161.1995.tb02031.x>
- Blei DM, Ng AY, Jordan MI (2003) Latent dirichlet allocation. *J Mach Learn Res* 3(null):993–1022
- Boyd RL (2022) *The Development and Psychometric Properties of LIWC-22*
- Champa AI, Rabbi MF, Nachuma C, Zibran MF (2024) Chatgpt in action: Analyzing its use in software development
- Chatterjee P, Gause B, Hedinger H, Pollock L (2017) Extracting code segments and their descriptions from research articles. In: *2017 IEEE/ACM 14th international conference on mining software repositories (MSR)*, pp 91–101. <https://doi.org/10.1109/MSR.2017.10>
- Chatterjee P, Kong M, Pollock L (2020) Finding help with programming errors: An exploratory study of novice software engineers' focus in stack overflow posts. *J Syst Softw* 159:110454
- Chatterjee P, Damevski K, Kraft NA, Pollock L (2021) Automatically identifying the quality of developer chats for post hoc use. *ACM Trans Softw Eng Methodol* 30(4). <https://doi.org/10.1145/3450503>
- Chen C, Xing Z, Wang X (2017) Unsupervised software-specific morphological forms inference from informal discussions. In: *2017 IEEE/ACM 39th international conference on software engineering (ICSE)*, pp 450–461. <https://doi.org/10.1109/ICSE.2017.48>
- Chen Z, Jiang L (2025) Evaluating software development agents: Patch patterns, code quality, and issue complexity in real-world github scenarios. In: *2025 IEEE international conference on software analysis, evolution and reengineering (SANER)*, pp 657–668. <https://doi.org/10.1109/SANER64311.2025.00068>
- Chouchen M, Bessghaier N, Begoug M, Ouni A, AlOmar EA, Wiem Mkaouer M (2024) How do software developers use chatgpt? an exploratory study on github pull requests. In: *2024 IEEE/ACM 21st international conference on mining software repositories (MSR)*, pp 212–216
- Clark HH, Brennan S (1991) Grounding in communication. In: *Perspectives on socially shared cognition*
- Cochran WG (1977) *Sampling Techniques*, 3rd edn. John Wiley & Sons, New York, NY
- Colavito G, Lanubile F, Novielli N, Quaranta L (2024) Leveraging gpt-like llms to automate issue labeling. In: *Proceedings of the 21st international conference on mining software repositories, association for computing machinery, New York, NY, USA, MSR '24*, p 469–480. <https://doi.org/10.1145/3643991.3644903>
- Cong G, Wang L, Lin CY, Song YI, Sun Y (2008) Finding question-answer pairs from online forums. In: *Proceedings of the 31st annual international ACM SIGIR conference on research and development in information retrieval, association for computing machinery, New York, NY, USA, SIGIR '08*, p 467–474. <https://doi.org/10.1145/1390334.1390415>

- Corbin J, Strauss A (2008) Basics of Qualitative Research, 3rd edn. SAGE Publications Inc, techniques and procedures for developing grounded theory
- Cordy JR, Roy CK (2011) The nicad clone detector. In: Proceedings of the 2011 IEEE 19th International Conference on Program Comprehension. IEEE Comput Soc, USA, ICPC '11, p 219–220. <https://doi.org/10.1109/ICPC.2011.26>
- Alencar da Costa D, Abebe SL, McIntosh S, Kulesza U, Hassan AE (2014) An empirical study of delays in the integration of addressed issues. In: 2014 IEEE international conference on software maintenance and evolution, pp 281–290. <https://doi.org/10.1109/ICSME.2014.50>
- Danescu-Niculescu-Mizil C, Gamon M, Dumais S (2011) Mark my words! linguistic style accommodation in social media. In: Proceedings of the 20th international conference on world wide web, association for computing machinery, New York, NY, USA, WWW '11, p 745–754. <https://doi.org/10.1145/1963405.1963509>
- Danial A (2021) cloc: v1.92. <https://doi.org/10.5281/zenodo.5760077>
- Das JK, Mondal S, Roy CK (2024) Investigating the utility of chatgpt in the issue tracking system: An exploratory study. [arXiv:2402.03735](https://arxiv.org/abs/2402.03735)
- Das JK, Mondal S, Roy CK (2025) Why do developers engage with chatgpt in issue-tracker? investigating usage and reliance on chatgpt-generated code
- Delile Z, Radel S, Godínez J, Engstrom G, Brucker T, Young K, Ghanavati S (2023) Evaluating privacy questions from stack overflow: Can chatgpt compete? In: 2023 IEEE 31st international requirements engineering conference workshops (REW), pp 239–244. <https://doi.org/10.1109/REW57809.2023.00048>
- Dhasade AB, Venigalla ASM, Chimalakonda S (2020) Towards prioritizing github issues. In: Proceedings of the 13th innovations in software engineering conference (formerly known as india software engineering conference), association for computing Machinery, New York, NY, USA, ISEC '20. <https://doi.org/10.1145/3385032.3385052>
- Duijn M, Kucera A, Bacchelli A (2015) Quality questions need quality code: Classifying code fragments on stack overflow. In: 2015 IEEE/ACM 12th working conference on mining software repositories
- Eastman D (2023) How conversational programming will democratize computing. <https://thenewstack.io/how-conversational-programming-will-democratize-computing>
- Ehsani R, Parra E, Haiduc S, Chatterjee P (2025a) Hierarchical knowledge injection for improving llm-based program repair. In: 2025 IEEE/ACM 40th international conference on automated software engineering (ASE) [arXiv:2506.24015](https://arxiv.org/abs/2506.24015)
- Ehsani R, Pathak S, Chatterjee P (2025b) Towards detecting prompt knowledge gaps for improved llm-guided issue resolution. In: 2025 IEEE/ACM 22nd international conference on mining software repositories (MSR), pp 699–711. <https://doi.org/10.1109/MSR66628.2025.00107>
- Eklund A, Forsman M (2022) Topic modeling by clustering language model embeddings: Human validation on an industry dataset. In: Li Y, Lazaridou A (eds) Proceedings of the 2022 conference on empirical methods in natural language processing: Industry track, association for computational linguistics, Abu Dhabi, UAE, pp 635–643. <https://doi.org/10.18653/v1/2022.emnlp-industry.65>
- Ertugrul Sagdic MRI Arda Bayram (2024) On the taxonomy of developers' discussion topics with chatgpt
- Fan A, Gokkaya B, Harman M, Lyubarskiy M, Sengupta S, Yoo S, Zhang JM (2023) Large language models for software engineering: Survey and open problems. [arXiv:2310.03533](https://arxiv.org/abs/2310.03533)
- França HL, Goseva-Popstojanova K, Teixeira C, Laranjeiro N (2025) GPTs are not the silver bullet: Performance and challenges of using GPTs for security bug report identification. *Inf Softw Technol* 185:107778. <https://doi.org/10.1016/j.infsof.2025.107778>
- GitHub (2023a) Bug fix example. <https://github.com/blonestar/wp-theme-vite-tailwind/issues/11>
- GitHub (2023b) Code comment example. <https://github.com/OpenVoiceOS/ovos-technical-manual/issues/4>
- GitHub (2023c) Code enhancements example. <https://github.com/karthik/gptel/issues/76>
- GitHub (2023d) Code explanation example. <https://github.com/LukeberryPi/phived/issues/55>
- GitHub (2023e) Code generation example. <https://github.com/akorb/SteamShutdown/issues/52>
- GitHub (2023f) Explicit positive comment example. <https://github.com/jasonacox/Powerwall-Dashboard/issues/288>
- GitHub (2023g) Explicit positive comment example. <https://github.com/WordPress/wordpress-playground/issues/1096>
- GitHub (2023h) Explicit positive comment example. <https://github.com/hackforla/website/issues/5086>
- GitHub (2023i) Failed answers in conversation with chatgpt. <https://github.com/swc-project/swc/issues/7754>
- GitHub (2023j) Generic answers in conversation with chatgpt. <https://github.com/christofmuc/KnobKraft-orm/issues/73>
- GitHub (2023k) Hallucinated answers in conversation with chatgpt. <https://github.com/chrisjpatty/flume/issues/237>
- GitHub (2023l) Hallucinated answers in conversation with chatgpt. <https://github.com/NCATSTranslator/Feedback/issues/198>

- GitHub (2023m) Helpful issue providing a link to chatgpt. <https://github.com/OpenAdaptAI/OpenAdapt/issues/570>
- GitHub (2023n) Information retrieval of chatgpt. <https://github.com/winglang/wing/issues/2869>
- GitHub (2023o) Information seeking example. <https://github.com/meixger/hackernews-daily/issues/251>
- GitHub (2023p) Irrelevant answers in conversation with chatgpt. <https://github.com/dubinc/dub/issues/479>
- GitHub (2023q) Knowledge seeking from chatgpt. <https://github.com/meta-llama/llama/issues/639>
- GitHub (2023r) Lsm example in conversation with chatgpt. <https://github.com/moses-palmer/pynput/issues/588>
- GitHub (2023s) Outdated answers in conversation with chatgpt. <https://github.com/nv-morpheus/Morpheus/issues/1210>
- GitHub (2023t) Semantic similarity example in conversation with chatgpt. <https://github.com/sikalabs/sluis/issues/235>
- GitHub (2023u) Shift in conversation with chatgpt. <https://github.com/udayhello/handwritten1/issues/2>
- GitHub (2023v) System design example. https://github.com/sky-unlimited/wef_2023/issues/26
- GitHub (2023w) Test generation example. <https://github.com/simonw/symbex/issues/18>
- GitHub (2023x) Tool/library example. <https://github.com/TryQuiet/quiet/issues/1495>
- GitHub (2023y) Toxic language using chatgpt. <https://github.com/sleuthkit/sleuthkit/issues/2872>
- GitHub (2023z) Trust in conversation with chatgpt. <https://github.com/winglang/wing/issues/2869>
- GitHub (2024) Octoverse 2023: The state of open source. <https://octoverse.github.com/>
- GitHubIssue (2023a) Example of positive sentiment conversation. <https://github.com/darklang/dark/issues/5314>
- GitHubIssue (2023b) Unclear answers in conversation with chatgpt. <https://github.com/webtorrent/webtorrent/issues/2536>
- GitHubIssue (2023c) Wrong answers in conversation with chatgpt. <https://github.com/kreait/firebase-php/issues/802>
- Han Y, Wang Z, Feng Y, Zhao Z, Wang Y (2024) Characterizing developers' linguistic behaviors in open source development across their social statuses. *Proc ACM Hum-Comput Interact* 8(CSCW1). <https://doi.org/10.1145/3637306>
- Hao H, Hasan KA, Qin H, Macedo M, Tian Y, Ding SHH, Hassan AE (2024) An empirical study on developers shared conversations with chatgpt in github pull requests and issues. 2403.10468
- Harper FM, Moy D, Konstan JA (2009) Facts or friends? distinguishing informational and conversational questions in social q&a sites. In: *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems*, Association for Computing Machinery, New York, NY, USA, CHI '09, p 759–768. <https://doi.org/10.1145/1518701.1518819>
- Hollander M, Wolfe DA, Chicken E (2013) *Nonparametric Statistical Methods*, 3rd edn. John Wiley & Sons
- Hossain SB, Jiang N, Zhou Q, Li X, Chiang WH, Lyu Y, Nguyen H, Tripp O (2024) A deep dive into large language models for automated bug localization and repair. *Proc ACM Softw Eng* 1(FSE). <https://doi.org/10.1145/3660773>
- Hou X, Zhao Y, Liu Y, Yang Z, Wang K, Li L, Luo X, Lo D, Grundy J, Wang H (2024) Large language models for software engineering: A systematic literature review. [arXiv:1203.81062](https://arxiv.org/abs/1203.81062)
- Imran MM, Jain Y, Chatterjee P, Damevski K (2022) Data augmentation for improving emotion recognition in software engineering communication. In: *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering*
- Islam MR, Zibran MF (2018) Sentistrength-se: Exploiting domain specificity for improved sentiment analysis in software engineering text. *J Syst Softw* 145:125–146. <https://doi.org/10.1016/j.jss.2018.08.030>
- Jimenez CE, Yang J, Wettig A, Yao S, Pei K, Press O, Narasimhan KR (2024) Swe-bench: Can language models resolve real-world github issues? In: *ICLR*, <https://openreview.net/forum?id=VTF8yNQm66>
- Jin K, Wang CY, Pham HV, Hemmati H (2024) Can ChatGPT support developers? An empirical evaluation of large language models for code generation. <https://doi.org/10.1145/3643991.3645074>, [arXiv:2402.11702](https://arxiv.org/abs/2402.11702) [cs]
- Kabir S, Udo-Imeh DN, Kou B, Zhang T (2024) Is stack overflow obsolete? An empirical study of the characteristics of chatgpt answers to stack overflow questions. [arXiv:2308.02312](https://arxiv.org/abs/2308.02312)
- Kavaler D, Sirovica S, Hellendoorn V, Aranovich R, Filkov V (2017) Perceived language complexity in github issue discussions and their effect on issue resolution. In: *2017 32nd IEEE/ACM international conference on automated software engineering (ASE)*, pp 72–81 <https://doi.org/10.1109/ASE.2017.8115620>
- Khandelwal T (2025) Using llm-based approaches to enhance and automate topic labeling. [arXiv:2502.18469](https://arxiv.org/abs/2502.18469)
- Li J, Mynatt ED, Mishra V, Bell J (2023) Always nice and confident, sometimes wrong: Developer's experiences engaging generative ai chatbots versus human-powered q&a platforms. *ArXiv abs/2309.13684*
- Liu D, Gong C, Zhang S, Ma Y (2022) The influence of firm's feedbacks on user-generated content's linguistic style matching—an explanation based on communication accommodation theory. *Frontiers Psychol* 13
- Liu F, Liu Y, Shi L, Huang H, Wang R, Yang Z, Zhang L, Li Z, Ma Y (2024) Exploring and evaluating hallucinations in llm-powered code generation. [arXiv:2404.00971](https://arxiv.org/abs/2404.00971)

- Liu J, Tang X, Li L, Chen P, Liu Y (2023) Which is a better programming assistant? a comparative study between chatgpt and stack overflow. [arXiv:2308.13851](https://arxiv.org/abs/2308.13851)
- McHugh M (2012) Interrater reliability: The kappa statistic. *Biochemia medica časopis Hrvatskoga društva medicinskih biokemičara / HDMB* 22:276–82
- Miller JK, Alexander TJ (2025) Human-interpretable clustering of short text using large language models. *Royal Soc Open Sci* 12(1). <https://doi.org/10.1098/rsos.241692>
- Mohamed S, Parvin A, Parra E (2024) Chatting with ai: Deciphering developer conversations with chatgpt
- Mondal S, Bappon SD, Roy CK (2024) Enhancing user interaction in chatgpt: Characterizing and consolidating multiple prompts for issue resolution. [arXiv:2402.04568](https://arxiv.org/abs/2402.04568)
- Nachar N (2008) The mann-whitney u: A test for assessing whether two independent samples come from the same distribution. *Tutorials in Quantitative Methods for Psychology* 4
- Novielli N, Calefato F, Dongiovanni D, Girardi D, Lanubile F (2020) Can we use se-specific sentiment analysis tools in a cross-platform setting? In: *Proceedings of the 17th International Conference on Mining Software Repositories*
- Oedingen M, Engelhardt RC, Denz R, Hammer M, Konen W (2024) Chatgpt code detection: Techniques for uncovering the source of code. *AI* 5(3). <https://doi.org/10.3390/ai5030053>
- OpenAI (2023) Chatgpt. <https://www.openai.com/>
- Pennebaker JW, Mehl MR, Niederhoffer KG (2003) Psychological aspects of natural language. Use: Our words, our selves. *Ann Rev Psychol* 54:547–577
- Ponzanelli L, Mocci A, Bacchelli A, Lanza M, Fullerton D (2014) Improving low quality stack overflow post detection. In: *2014 IEEE international conference on software maintenance and evolution*, pp 541–544. <https://doi.org/10.1109/ICSME.2014.90>
- Possel H (2024) 50 Popular texting abbreviations & internet acronyms. <https://www.smart-words.org/abbreviations/text.html>
- Qian C, Zhang M, Nie Y, Lu S, Cao H (2023) A survey on bug deduplication and triage methods from multiple points of view. *Applied Sciences* 13(15). <https://doi.org/10.3390/app13158788>
- Reimers N, Gurevych I (2024) all-minilm-l6-v2. <https://huggingface.co/sentence-transformers/all-MiniLM-L6-v2>
- Ren Y, Gay G, Kästner C, Jamshidi P (2020) Understanding the nature of system-related issues in machine learning frameworks: An exploratory study. [arXiv:2005.06091](https://arxiv.org/abs/2005.06091)
- Rigby PC, Hassan AE (2007) What can oss mailing lists tell us? a preliminary psychometric text analysis of the apache developer mailing list. In: *Fourth International Workshop on Mining Software Repositories (MSR'07:ICSE Workshops 2007)*, pp 23–23. <https://doi.org/10.1109/MSR.2007.35>
- Röder M, Both A, Hinneburg A (2015) Exploring the space of topic coherence measures. In: *Proceedings of the 8th ACM international conference on web search and data mining, association for computing machinery, New York, NY, USA, WSDM '15*, p 399–408
- Ross SI, Martinez F, Houde S, Muller M, Weisz JD (2023) The programmer's assistant: Conversational interaction with a large language model for software development. In: *Proceedings of the 28th international conference on intelligent user interfaces, ACM, IUI '23*
- Sandler M, Choung H, Ross A, David P (2024) A linguistic comparison between human and chatgpt-generated conversations. [arXiv:2401.16587](https://arxiv.org/abs/2401.16587)
- Schäfer M, Nadi S, Eghbali A, Tip F (2024) An empirical evaluation of using large language models for automated unit test generation. *IEEE Trans Software Eng* 50(1):85–105. <https://doi.org/10.1109/TSE.2023.3334955>
- Scott B (2023) The SMOG Readability Formula, a Simple Measure of Gobbledygook. <https://readabilityformulas.com/the-smog-readability-formula/>, Section: Readability Formulas Help
- Shuster K, Poff S, Chen M, Kiela D, Weston J (2021) Retrieval augmentation reduces hallucination in conversation. In: Moens MF, Huang X, Specia L, Yih SWt (eds) *Findings of the association for computational linguistics: EMNLP 2021, association for computational linguistics, Punta Cana, Dominican Republic*, pp 3784–3803. <https://doi.org/10.18653/v1/2021.findings-emnlp.320>
- Silva LD, Samhi J, Khomh F (2024) Chatgpt vs llama: Impact, reliability, and challenges in stack overflow discussions. [arXiv:2402.08801](https://arxiv.org/abs/2402.08801)
- spaCy (2024) spacy PyPI. <https://pypi.org/project/spacy/>
- Stahl PM (2024) pemistahl/lingua-py. <https://github.com/pemistahl/lingua-py>
- Tan Z, Li D, Wang S, Beigi A, Jiang B, Bhattacharjee A, Karami M, Li J, Cheng L, Liu H (2024) Large language models for data annotation and synthesis: A survey. [arXiv:2402.13446](https://arxiv.org/abs/2402.13446)
- Tang N, Chen M, Ning Z, Bansal A, Huang Y, McMillan C, Li TJJ (2023) An empirical study of developer behaviors for validating and repairing ai-generated code. *Plateau Workshop*
- Tao W, Zhou Y, Wang Y, Zhang W, Zhang H, Cheng Y (2024) Magis: Llm-based multi-agent framework for github issue resolution. [arXiv:2403.17927](https://arxiv.org/abs/2403.17927)
- TechTerms (2024) Dictionary of Software Terms. <https://techterms.com/category/software>

- Umargono E, Suseno JE, Gunawan SV (2020) K-means clustering optimization using the elbow method and early centroid determination based on mean and median formula. In: Proceedings of the 2nd international seminar on science and technology (ISSTEC 2019), Atlantis Press, pp 121–129. <https://doi.org/10.2991/assehr.k.201010.019>
- Vakulenko S, de Rijke M, Cochez M, Savenkov V, Polleres A (2018) Measuring semantic coherence of a conversation. [arXiv:1806.06411](https://arxiv.org/abs/1806.06411)
- Wang Q, Saha K, Gregori E, Joyner D, Goel A (2021) Towards mutual theory of mind in human-ai interaction: How language reflects what students perceive about a virtual teaching assistant. In: Proceedings of the 2021 CHI conference on human factors in computing systems, association for computing machinery, New York, NY, USA, CHI '21
- Wu Y, Li Z, Zhang JM, Papadakis M, Harman M, Liu Y (2023) Large language models in fault localisation. [arXiv:2308.15276](https://arxiv.org/abs/2308.15276)
- Xiao T, Treude C, Hata H, Matsumoto K (2024) DevGPT: Studying Developer-ChatGPT Conversations. <https://doi.org/10.1145/3643991.3648400>
- Xu B, Nguyen TD, Le-Cong T, Hoang T, Liu J, Kim K, Gong C, Niu C, Wang C, Le B, Lo D (2023) Are we ready to embrace generative ai for software q&a? In: 2023 38th IEEE/ACM international conference on automated software engineering (ASE), pp 1713–1717. <https://doi.org/10.1109/ASE56229.2023.00023>
- Yang J, Jin H, Tang R, Han X, Feng Q, Jiang H, Zhong S, Yin B, Hu X (2024) Harnessing the power of llms in practice: A survey on chatgpt and beyond. *ACM Trans Knowl Discov Data* 18(6)
- Zhang T, Xu B, Thung F, Haryono SA, Lo D, Jiang L (2020) Sentiment analysis for software engineering: How far can pre-trained transformer models go? In: 2020 IEEE international conference on software maintenance and evolution (ICSME), pp 70–80. <https://doi.org/10.1109/ICSME46990.2020.00017>
- Zhang Y, Ruan H, Fan Z, Roychoudhury A (2024) Autocoderover: Autonomous program improvement. [arXiv:2404.05427](https://arxiv.org/abs/2404.05427)
- Zhou J, Zhang Y, Luo Q, Parker AG, De Choudhury M (2023) Synthetic lies: Understanding ai-generated misinformation and evaluating algorithmic and human solutions. In: Proceedings of the 2023 CHI conference on human factors in computing systems, association for computing machinery, New York, NY, USA, CHI '23. <https://doi.org/10.1145/3544548.3581318>,
- Zhou X, Liang P, Zhang B, Li Z, Ahmad A, Shahin M, Waseem M (2024) Exploring the problems, their causes and solutions of ai pair programming: A study with practitioners of github copilot. [arXiv:2311.01020](https://arxiv.org/abs/2311.01020)

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Authors and Affiliations

Ramtin Ehsani¹  · Sakshi Pathak¹ · Esteban Parra² · Sonia Haiduc³ · Preetha Chatterjee¹

✉ Ramtin Ehsani
ramtin.ehsani@drexel.edu

Sakshi Pathak
sp3856@drexel.edu

Esteban Parra
esteban.parrarodriguez@belmont.edu

Sonia Haiduc
shaiduc@cs.fsu.edu

Preetha Chatterjee
preetha.chatterjee@drexel.edu

¹ Drexel University, Philadelphia, Pennsylvania, USA

² Belmont University, Nashville, Tennessee, USA

³ Florida State University, Tallahassee, Florida, USA